uc-se-sm26-team1 / scrum-project

Code Issues 11 Pull requests Agents Actions Projects Security and quality Insights

Settings

main scrum-project / sprint2 / README.md

pawanpoudyel Fix text 926f4ad · 7 minutes ago

848 lines (615 loc) · 52.6 KB

Messenger v0.2 — Final Scrum Project Report

main scrum-project / sprint2 / README.md Top

Preview Code Blame Raw

Instructor: Dr. Phu Phung

Team: UC Software Engineering Summer 2026, Team 1

Team Members

Member	University email	Scrum role
Corey Brunner	brunnecj@mail.uc.edu	Scrum Master
Pawan Poudyel	poudyepn@mail.uc.edu	Product Owner
Zackary Bailey	baileyzc@mail.uc.edu	Developer
Abdulah Dia	diaah@mail.uc.edu	Developer
Garrett Becker	beckergt@mail.uc.edu	Developer

Project Management Information

Item	Link or configuration
Team homepage	https://uc-se-sm26-team1.github.io/
Team homepage repository	uc-se-sm26-team1.github.io
Live Messenger v0.2 prototype	Azure App Service
GitHub Projects board	Team 1 Scrum project
Private source repository	uc-se-sm26-team1/scrum-project
MongoDB Atlas	`Team1Messenger` cluster, Atlas free tier; credentials are supplied only through `MONGO_URI`

The team homepage and live Azure URL returned HTTP 200 during the documentation audit on August 4, 2026. The Azure response also included the application's Content Security Policy header.

Revision History

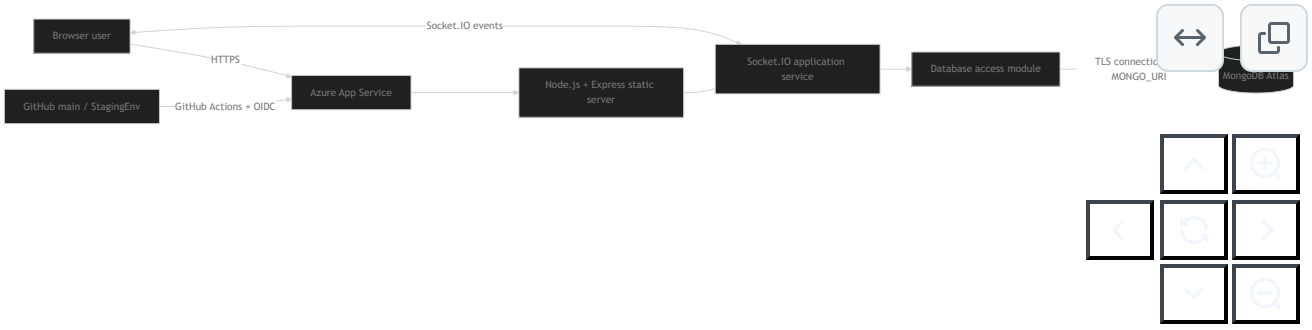
Date	Version	Description	Author
2026-05-28	0.0	Created the repository, team information, links, and initial report structure	Team 1
2026-06-17	0.0	Completed Sprint 0 requirements, use cases, threat model, architecture, and retrospective	Team 1
2026-07-09	0.1	Documented the first working public/private real-time chat prototype and Azure pipeline	Team 1
2026-07-30	0.2	Added database-backed accounts, chat history, profile management, logout, group chat, and message deletion	Team 1
2026-08-04	0.2	Audited the report against the final rubric and synchronized it with the implemented source	Team 1, AI-assisted

Overview

Messenger is a real-time web application for students, faculty, and project teams that need one browser-based space for public, private, and group communication. It reduces the need to switch between incompatible desktop applications and gives connected users immediate messages, typing indicators, online-presence information, clickable links, and access to public chat history.

Version 0.2 replaces Sprint 1's username-only entry with accounts stored in MongoDB Atlas. A visitor can register with a username, password, email address, and full name; authenticate with a username and password; update profile information; and log out. Passwords are hashed with bcrypt before storage. Public messages are persisted and restored after login, and a sender can soft-delete their own public message. The final client also includes private messaging, name-based group rooms, public profiles for online users, and separate public/private/group panels.

The application uses a client-server architecture. Azure App Service hosts a Node.js/Express application that serves the single-page HTML/CSS/JavaScript client. Socket.IO carries authentication and real-time events. A dedicated database module reads and writes users and public message history in MongoDB Atlas. GitHub Actions builds an artifact and deploys approved branch changes to staging or production.



System Analysis

Sprint Evolution

Sprint 0 — analysis and design. The team established personas and scenarios, selected a browser-based Messenger product, created the initial use cases and acceptance criteria, modeled security threats, and produced architecture and interface designs.

Sprint 1 — Messenger v0.1. The team implemented separate login and chat views, username-based entry, public and private real-time messaging, a live online-user list, separate public/private panels, typing indicators, automatic scrolling, link insertion, and automatic conversion of HTTP(S) URLs. The prototype ran on Node.js and Socket.IO and was deployed through GitHub Actions to Azure App Service.

Sprint 2 — Messenger v0.2. The team connected the server to MongoDB Atlas and replaced username-only entry with registration and password authentication. Sprint 2 also introduced bcrypt password hashes, multi-layer registration validation, authenticated Socket.IO operations, logout, account editing, public chat persistence, owner-only soft deletion, online-user profiles, name-based group rooms, a redesigned Bootstrap-based login/registration interface, staging deployment, CSP response headers, and environment-based secret configuration.

User Requirements

Required Functional Requirements and Traceability

ID	Final requirement	v0.2 realization and evidence
F1.1	Login and chat UIs are visually and functionally separated.	`#loginUI`, `#registerUI`, and `#chatUI` are separate sections in `index.html`; only the appropriate view is shown.
F1.2	A v0.1 user can enter with a username and no password.	Delivered in Sprint 1 and intentionally superseded by the password requirement in F2.1 for v0.2.
F1.3	Only logged-in users can access the chat interface.	The client reveals the chat only after `join-success`; the server independently applies `authorizeUser()` to protected events.
F1.4	Logged-in users can send and receive public messages in real time.	Socket.IO `message` events persist through `storePublicChat()` and broadcast to authenticated sockets.
F1.5	Logged-in users can send and receive private messages with a specific online user.	`private-message` validates a recipient, locates that user's socket, and sends only to the recipient and sender.
F1.6	An online-user list is displayed and updated in real time.	The server maintains a socket-to-username map and emits `user-list` after login, logout, and disconnect.

ID	Final requirement	v0.2 realization and evidence
F1.7	Public and private chat are visually distinct.	The client has separate labeled public, private, and group panels and separate response areas.
F1.8	Public and private typing indicators update in real time.	`public-typing`, `public-stop-typing`, `private-typing`, and `private-stop-typing` events update dedicated UI elements.
F1.9	Team feature for v0.1: external website links.	HTTP(S) text is linkified after paste or a 500 ms typing delay; an insert-link dialog supports custom display text.
F2.1	Users authenticate with a database-backed username and password; invalid credentials fail.	`join-chat` calls `messengerdb.find()`, which looks up the username and uses `bcrypt.compare()` against the stored hash.
F2.2	Anyone can register with username/password and additional profile fields.	`register` collects username, password, email, and full name, validates them at three layers, hashes the password, and inserts a user document.
F2.3	Only logged-in users can send or receive application messages.	Protected server events call `authorizeUser()` and broadcasts use `sendToAuthenticatedClients()`.
F2.4	A logged-in user can log out.	`logout` clears authentication, removes the user from groups and the online list, then returns the browser to the login UI.
F2.5	A logged-in user can edit their profile, including password.	`edit-profile` requires the current password and can update the bcrypt password hash, email, and full name.
F2.6	Messages are stored in the database and history is visible after login.	The latest 100 public messages are stored in `MessageHistory`, sorted by timestamp, and emitted as `chat_history`. Private and group history is not persisted in v0.2.

ID	Final requirement	v0.2 realization and evidence
F2.7	Team feature for v0.2: delete an owned public message.	A sender can select `x`; the data layer verifies ownership and soft-deletes the message by replacing its content and setting `deleted: true`. The server should be hardened so it broadcasts `message-delete` only after a successful owner-authorized update.
F2.SEC	Password hashing is verifiable in the database.	Registration and password changes call `bcrypt.hash(value, 10)`; login calls `bcrypt.compare()`. Plaintext passwords are not written to MongoDB.

Non-Functional Requirements

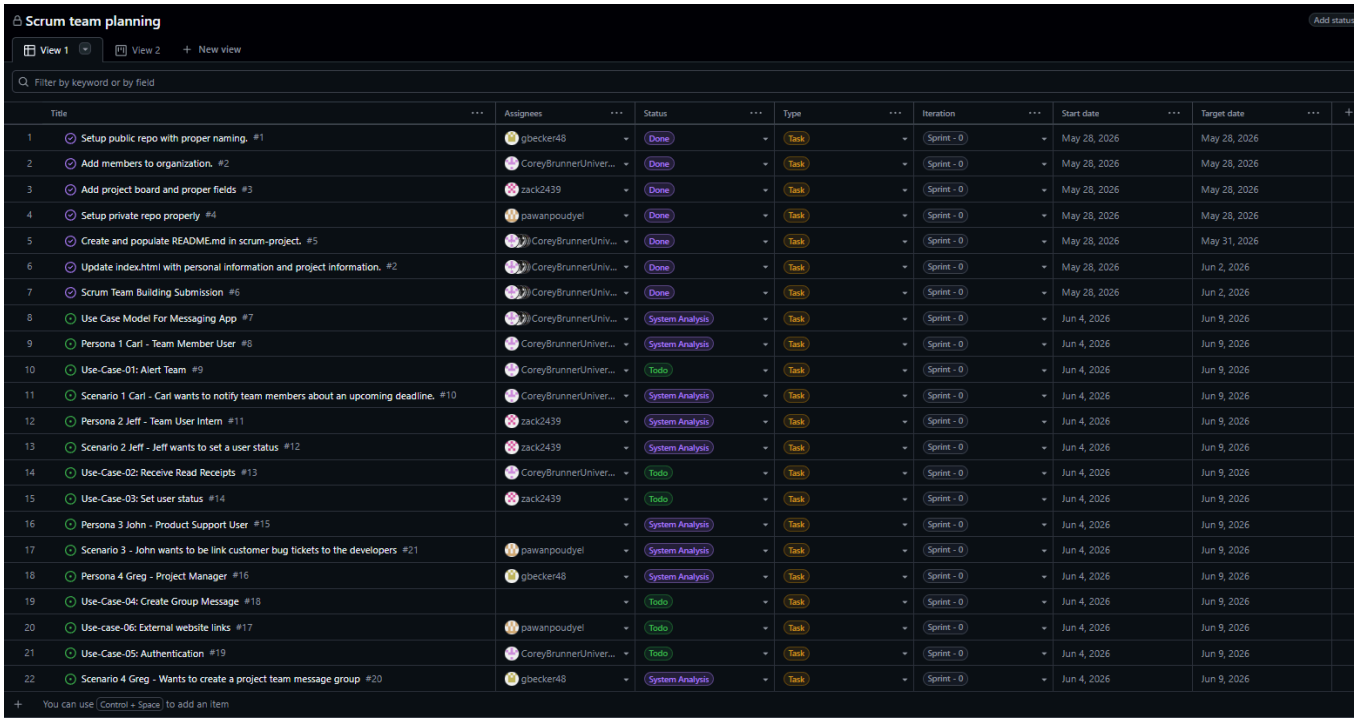
ID	Category	Requirement	Evidence/status
NFR-1	Performance	Pasted links shall be recognized immediately and typed links within one second under normal conditions.	Client linkification uses a 0 ms paste delay and 500 ms typing delay.
NFR-2	Usability	Login, registration, public chat, private chat, group chat, profile editing, and logout shall be clearly labeled.	Separate sections, panel headings, action buttons, inline validation, and success/error messages are present.
NFR-3	Security	Credentials shall use HTTPS in production and passwords shall never be persisted in plaintext.	The production URL uses HTTPS; bcrypt hashing and environment-based MongoDB credentials are implemented.
NFR-4	Data availability	A newly authenticated user shall receive recent public history without manually refreshing.	The server loads up to 100 messages at successful login.
NFR-5	Maintainability	Secrets and environment-specific configuration shall	`.env` is ignored, `.env.example` documents `MONGO_URI`, and Azure

ID	Category	Requirement	Evidence/status
		remain outside version-controlled source.	configuration supplies the production value.

User Stories and Product Backlog

The authoritative backlog is the private GitHub Projects board. Issues contain the user stories, acceptance-criteria codes, assignees, iteration dates, and progress state. The captures below preserve the board state for each sprint.

Sprint 0 Board



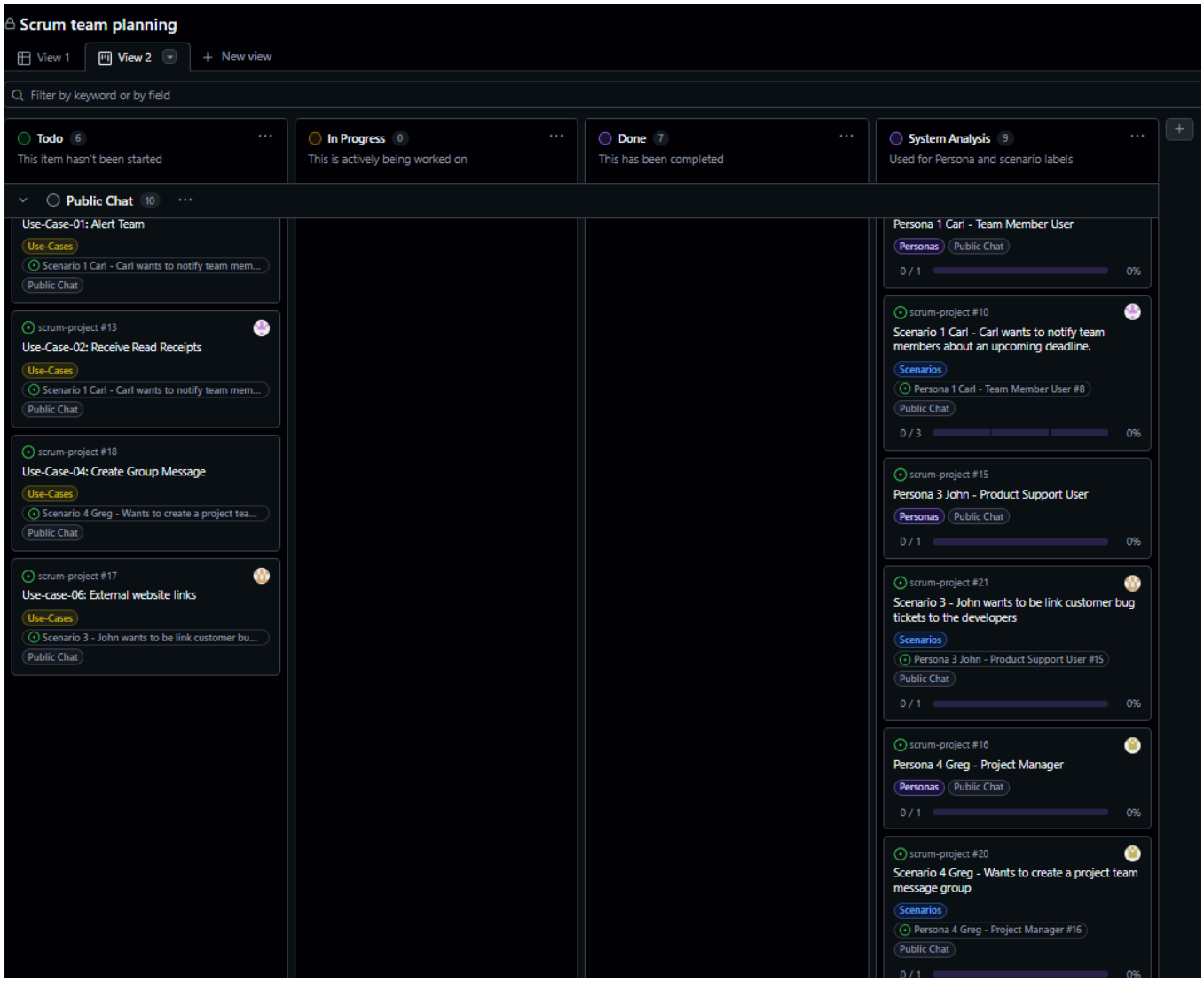
Scrum team planning

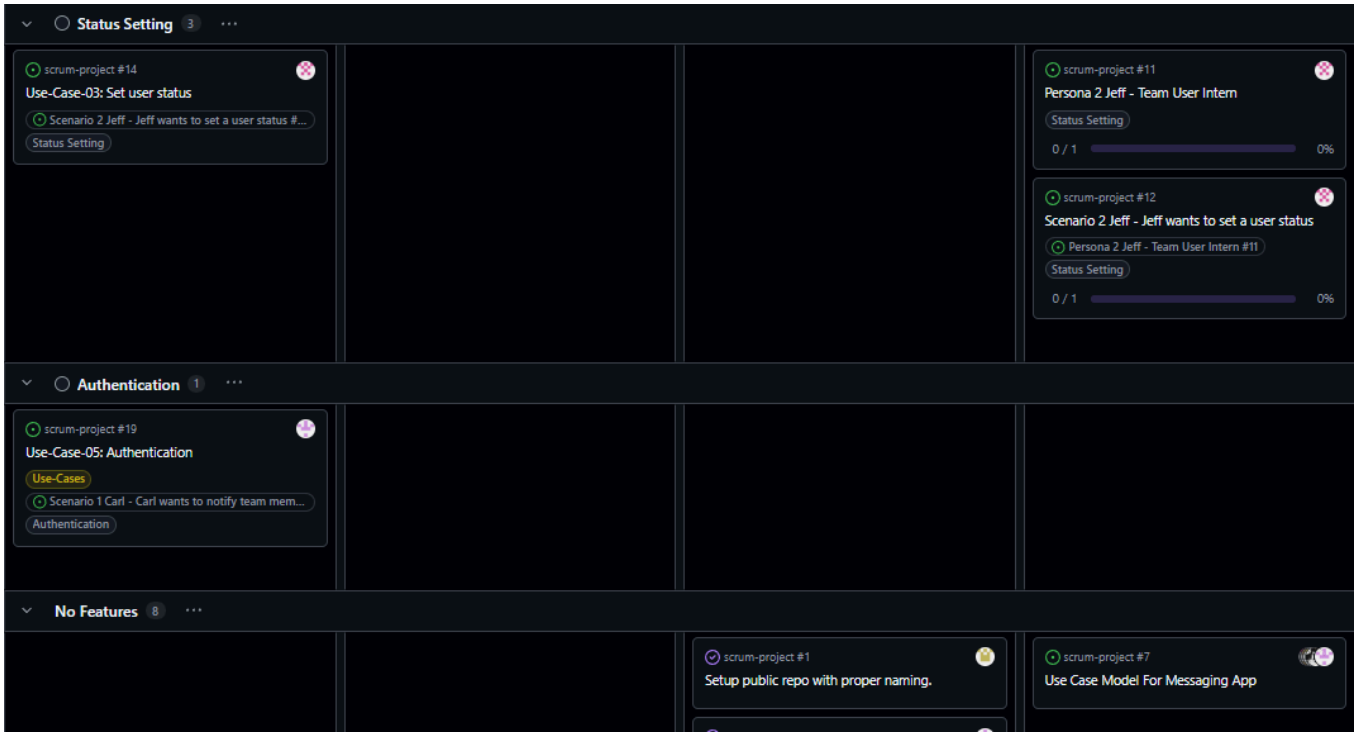
View 1 | View 2 | + New view

Filter by keyword or by field

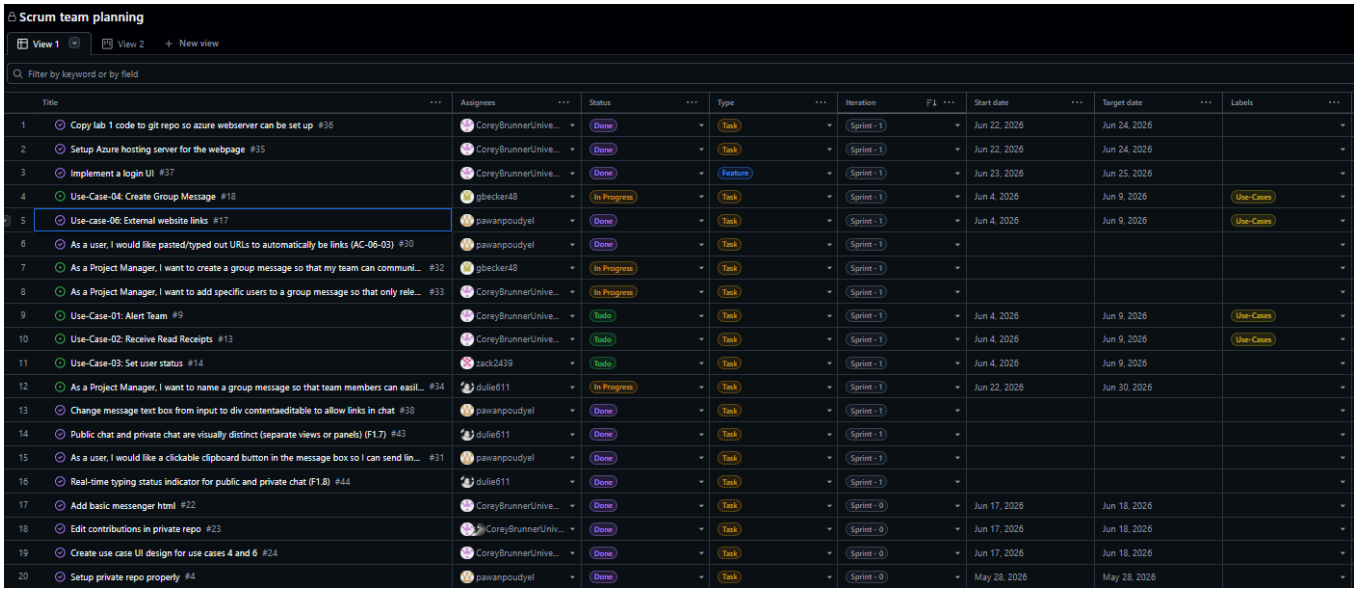
	Title	Assignees	Status	Type	Iteration	Start date	Target date
1	Setup public repo with proper naming. #1	gbecker48	Done	Task	Sprint - 0	May 28, 2026	May 28, 2026
2	Add members to organization. #2	CoreyBrunnerUniver...	Done	Task	Sprint - 0	May 28, 2026	May 28, 2026
3	Add project board and proper fields #3	zack2439	Done	Task	Sprint - 0	May 28, 2026	May 28, 2026
4	Setup private repo properly #4	pawanpoudyel	Done	Task	Sprint - 0	May 28, 2026	May 28, 2026
5	Create and populate README.md in scrum-project. #5	CoreyBrunnerUniv...	Done	Task	Sprint - 0	May 28, 2026	May 31, 2026
6	Update index.html with personal information and project information. #2	CoreyBrunnerUniv...	Done	Task	Sprint - 0	May 28, 2026	Jun 2, 2026
7	Scrum Team Building Submission #6	CoreyBrunnerUniv...	Done	Task	Sprint - 0	May 28, 2026	Jun 2, 2026
8	Use Case Model For Messaging App #7	CoreyBrunnerUniv...	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
9	Persona 1 Carl - Team Member User #8	CoreyBrunnerUniver...	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
10	Use-Case-01: Alert Team #9	CoreyBrunnerUniver...	Todo	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
11	Scenario 1 Carl - Carl wants to notify team members about an upcoming deadline. #10	CoreyBrunnerUniver...	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
12	Persona 2 Jeff - Team User Intern #11	zack2439	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
13	Scenario 2 Jeff - Jeff wants to set a user status #12	zack2439	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
14	Use-Case-02: Receive Read Receipts #13	CoreyBrunnerUniver...	Todo	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
15	Use-Case-03: Set user status #14	zack2439	Todo	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
16	Persona 3 John - Product Support User #15		System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
17	Scenario 3 - John wants to be link customer bug tickets to the developers #21	pawanpoudyel	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
18	Persona 4 Greg - Project Manager #16	gbecker48	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
19	Use-Case-04: Create Group Message #18		Todo	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
20	Use-case-06: External website links #17	pawanpoudyel	Todo	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
21	Use-Case-05: Authentication #19	CoreyBrunnerUniver...	Todo	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026
22	Scenario 4 Greg - Wants to create a project team message group #20	gbecker48	System Analysis	Task	Sprint - 0	Jun 4, 2026	Jun 9, 2026

+ You can use Control + Space to add an item


Scrum team planning
View 1
View 2
New view
Filter by keyword or by field
Todo 6
This item hasn't been started
In Progress 0
This is actively being worked on
Done 7
This has been completed
System Analysis 9
Used for Persona and scenario labels
Public Chat 10
Use-Case-01: Alert Team
Use-Cases
Scenario 1 Carl - Carl wants to notify team mem...
Public Chat
scrum-project #13
Use-Case-02: Receive Read Receipts
Use-Cases
Scenario 1 Carl - Carl wants to notify team mem...
Public Chat
scrum-project #18
Use-Case-04: Create Group Message
Use-Cases
Scenario 4 Greg - Wants to create a project tea...
Public Chat
scrum-project #17
Use-case-06: External website links
Use-Cases
Scenario 3 - John wants to be link customer bu...
Public Chat
Persona 1 Carl - Team Member User
Personas
Public Chat
0 / 1
0%
scrum-project #10
Scenario 1 Carl - Carl wants to notify team members about an upcoming deadline.
Scenarios
Persona 1 Carl - Team Member User #8
Public Chat
0 / 3
0%
scrum-project #15
Persona 3 John - Product Support User
Personas
Public Chat
0 / 1
0%
scrum-project #21
Scenario 3 - John wants to be link customer bug tickets to the developers
Scenarios
Persona 3 John - Product Support User #15
Public Chat
0 / 1
0%
scrum-project #16
Persona 4 Greg - Project Manager
Personas
Public Chat
0 / 1
0%
scrum-project #20
Scenario 4 Greg - Wants to create a project team message group
Scenarios
Persona 4 Greg - Project Manager #16
Public Chat


Status Setting 3
scrum-project #14
Use-Case-03: Set user status
Scenario 2 Jeff - Jeff wants to set a user status #...
Status Setting
scrum-project #11
Persona 2 Jeff - Team User Intern
Status Setting
0 / 1
0%
scrum-project #12
Scenario 2 Jeff - Jeff wants to set a user status
Persona 2 Jeff - Team User Intern #11
Status Setting
0 / 1
0%
Authentication 1
scrum-project #19
Use-Case-05: Authentication
Use-Cases
Scenario 1 Carl - Carl wants to notify team mem...
Authentication
No Features 8
scrum-project #1
Setup public repo with proper naming.
scrum-project #7
Use Case Model For Messaging App

Sprint 1 Board

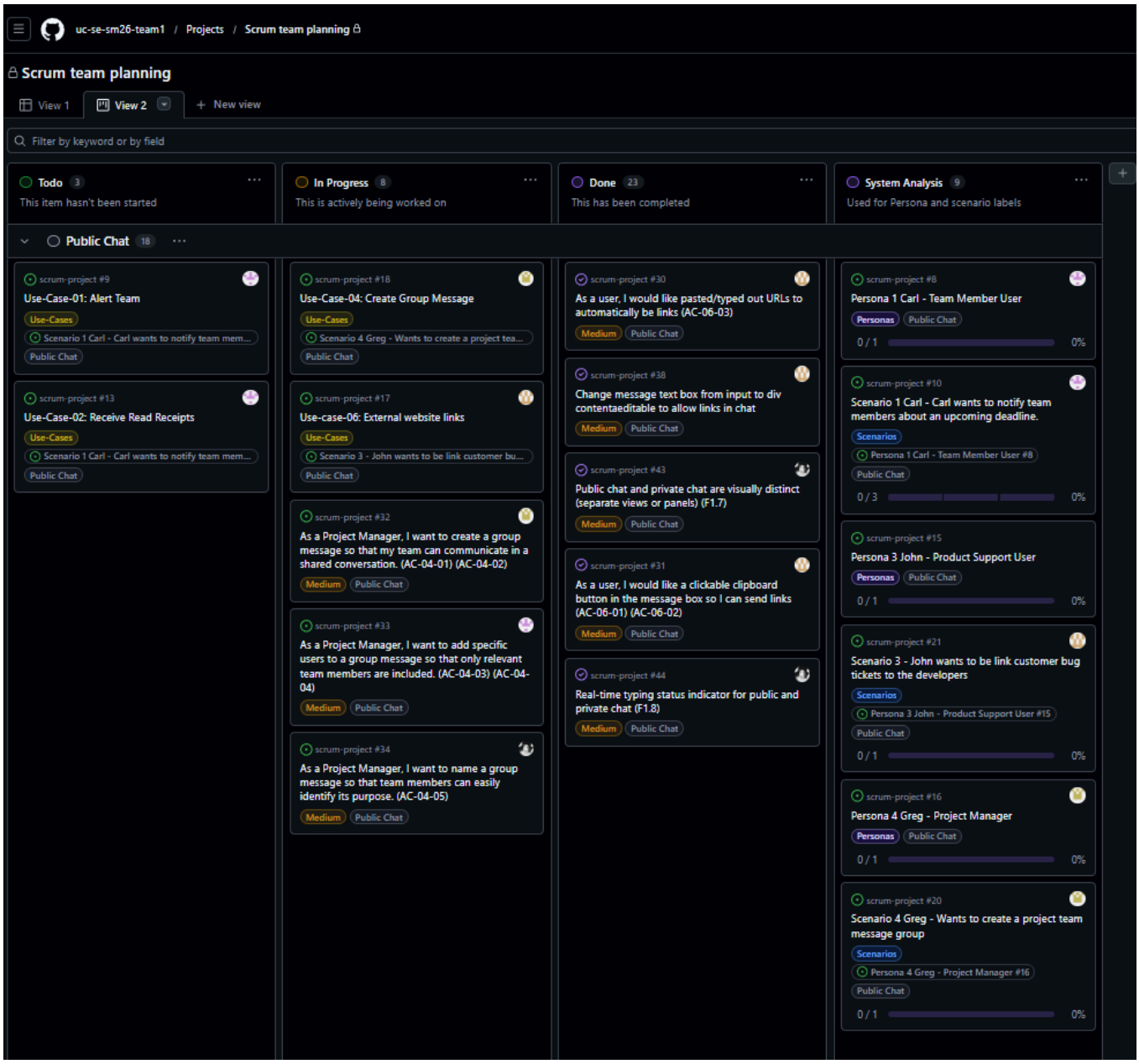


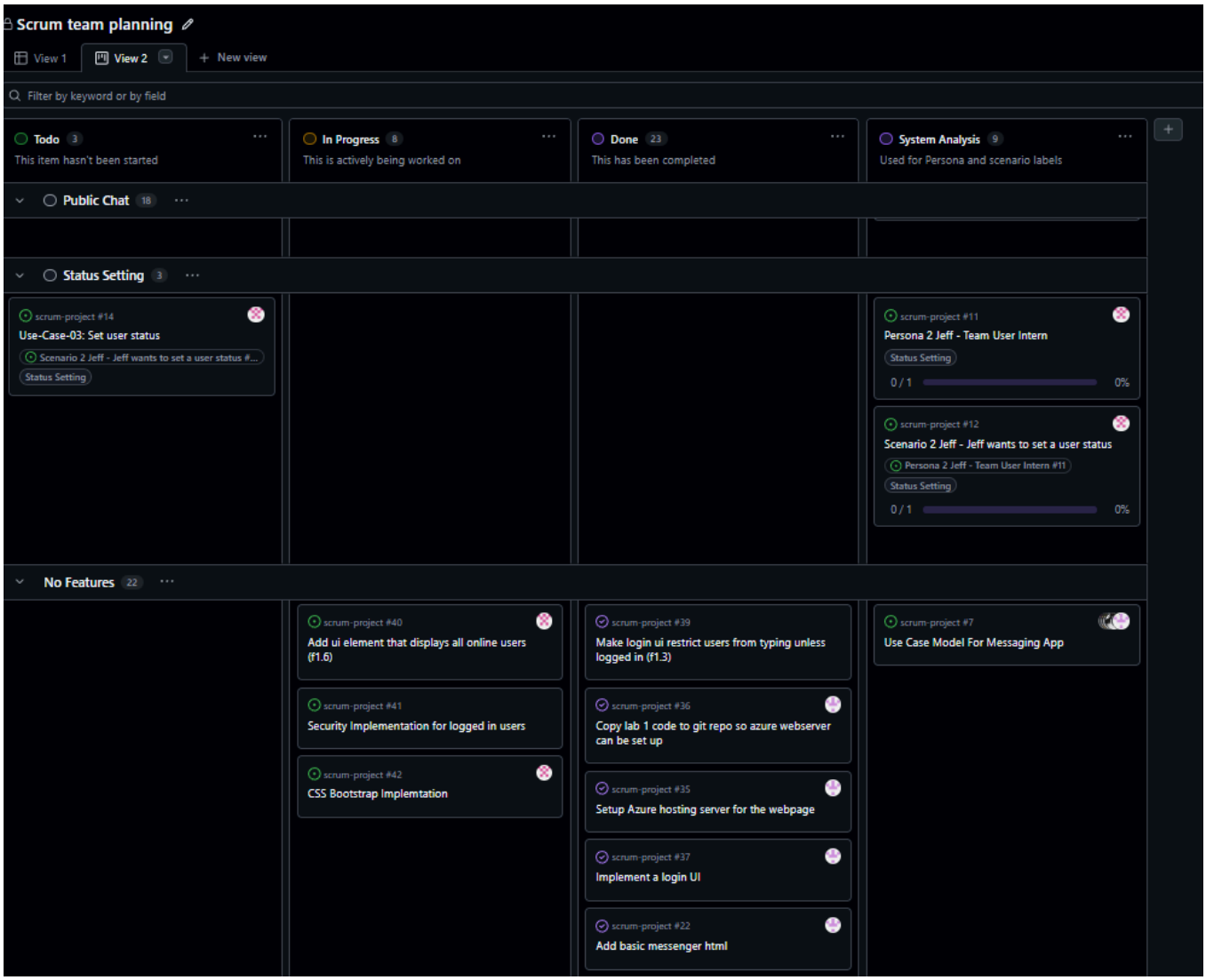
Scrum team planning

View 1 | View 2 | + New view

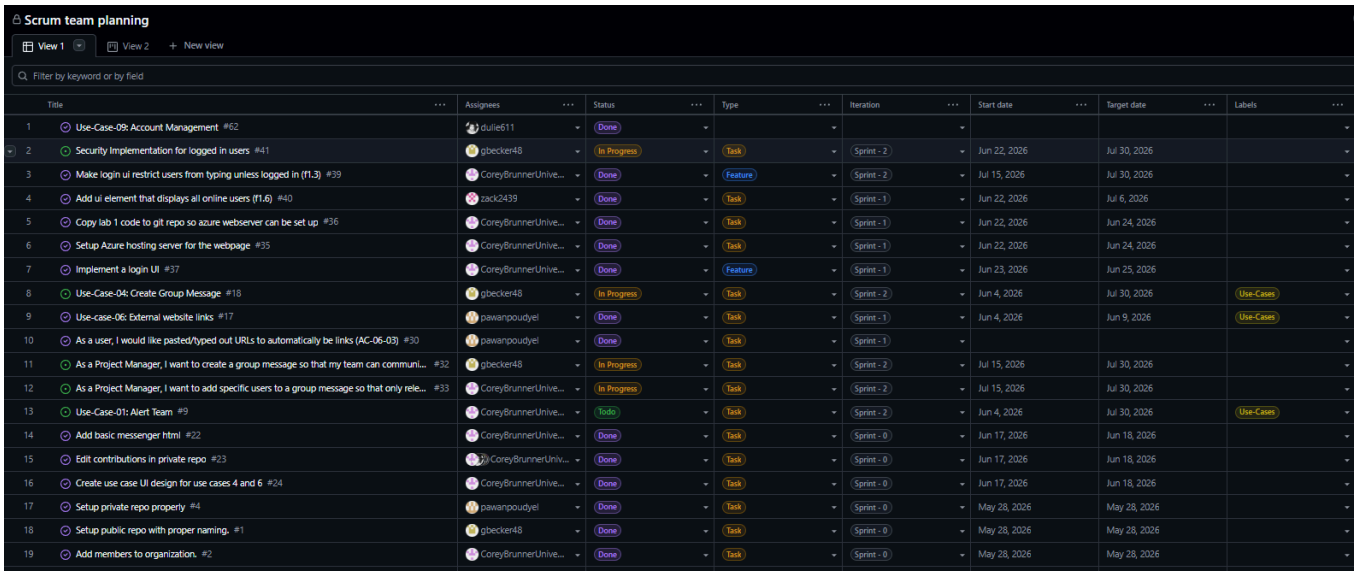
Filter by keyword or by field

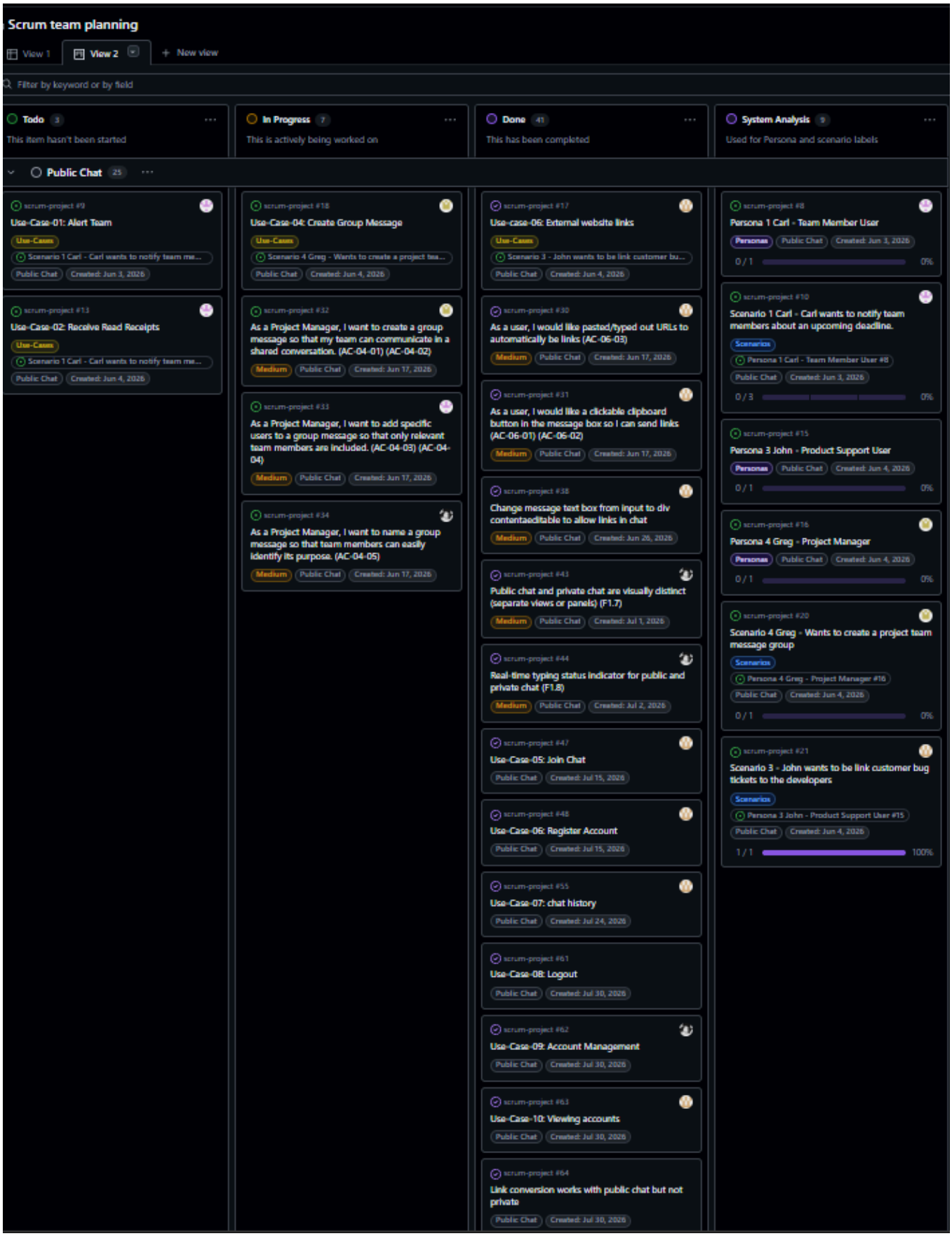
	Title	Assignees	Status	Type	Iteration	Start date	Target date	Labels
1	Copy lab 1 code to git repo so azure webserver can be set up #36	CoreyBrunnerUnive...	Done	Task	Sprint - 1	Jun 22, 2026	Jun 24, 2026	
2	Setup Azure hosting server for the webpage #35	CoreyBrunnerUnive...	Done	Task	Sprint - 1	Jun 22, 2026	Jun 24, 2026	
3	Implement a login UI #37	CoreyBrunnerUnive...	Done	Feature	Sprint - 1	Jun 23, 2026	Jun 25, 2026	
4	Use-Case-04: Create Group Message #18	gbecker48	In Progress	Task	Sprint - 1	Jun 4, 2026	Jun 9, 2026	Use-Cases
5	Use-case-06: External website links #17	pawanpoudyel	Done	Task	Sprint - 1	Jun 4, 2026	Jun 9, 2026	Use-Cases
6	As a user, I would like pasted/typed out URLs to automatically be links (AC-06-03) #30	pawanpoudyel	Done	Task	Sprint - 1			
7	As a Project Manager, I want to create a group message so that my team can communi... #32	gbecker48	In Progress	Task	Sprint - 1			
8	As a Project Manager, I want to add specific users to a group message so that only rele... #33	CoreyBrunnerUnive...	In Progress	Task	Sprint - 1			
9	Use-Case-01: Alert Team #9	CoreyBrunnerUnive...	Todo	Task	Sprint - 1	Jun 4, 2026	Jun 9, 2026	Use-Cases
10	Use-Case-02: Receive Read Receipts #13	CoreyBrunnerUnive...	Todo	Task	Sprint - 1	Jun 4, 2026	Jun 9, 2026	Use-Cases
11	Use-Case-03: Set user status #14	zack2439	Todo	Task	Sprint - 1	Jun 4, 2026	Jun 9, 2026	
12	As a Project Manager, I want to name a group message so that team members can easil... #34	dulie611	In Progress	Task	Sprint - 1	Jun 22, 2026	Jun 30, 2026	
13	Change message text box from input to div contentaeditable to allow links in chat #38	pawanpoudyel	Done	Task	Sprint - 1			
14	Public chat and private chat are visually distinct (separate views or panels) (F1.7) #43	dulie611	Done	Task	Sprint - 1			
15	As a user, I would like a clickable clipboard button in the message box so I can send lin... #31	pawanpoudyel	Done	Task	Sprint - 1			
16	Real-time typing status indicator for public and private chat (F1.8) #44	dulie611	Done	Task	Sprint - 1			
17	Add basic messenger html #22	CoreyBrunnerUnive...	Done	Task	Sprint - 0	Jun 17, 2026	Jun 18, 2026	
18	Edit contributions in private repo #23	CoreyBrunnerUniv...	Done	Task	Sprint - 0	Jun 17, 2026	Jun 18, 2026	
19	Create use case UI design for use cases 4 and 6 #24	CoreyBrunnerUnive...	Done	Task	Sprint - 0	Jun 17, 2026	Jun 18, 2026	
20	Setup private repo properly #4	pawanpoudyel	Done	Task	Sprint - 0	May 28, 2026	May 28, 2026	

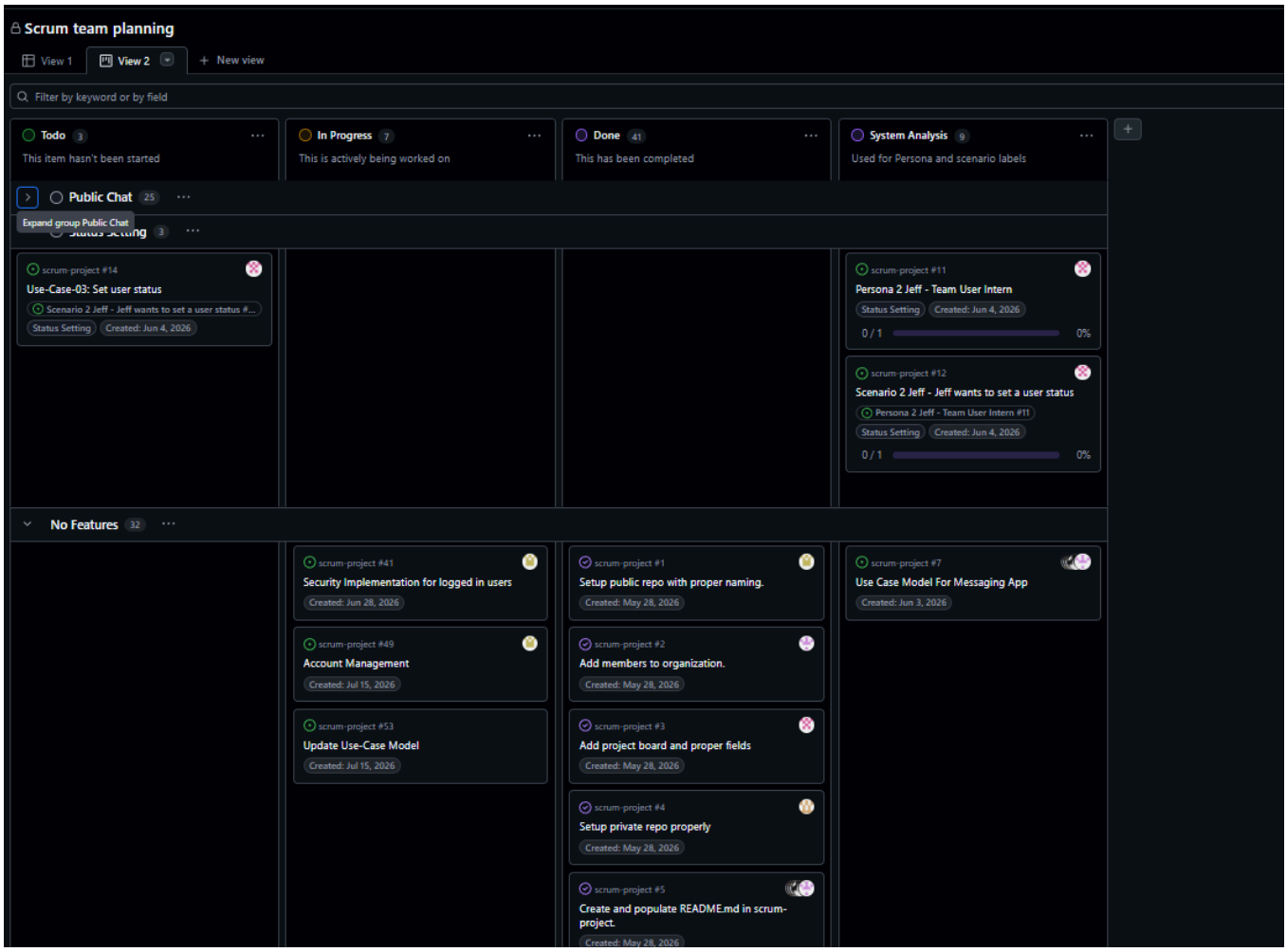

uc-se-sm26-team1 / Projects / Scrum team planning
Scrum team planning
View 1
View 2
New view
Filter by keyword or by field
Todo 3
This item hasn't been started
In Progress 8
This is actively being worked on
Done 23
This has been completed
System Analysis 9
Used for Persona and scenario labels
Public Chat 18
scrum-project #9
Use-Case-01: Alert Team
Use-Cases
Scenario 1 Carl - Carl wants to notify team mem...
Public Chat
scrum-project #13
Use-Case-02: Receive Read Receipts
Use-Cases
Scenario 1 Carl - Carl wants to notify team mem...
Public Chat
scrum-project #18
Use-Case-04: Create Group Message
Use-Cases
Scenario 4 Greg - Wants to create a project tea...
Public Chat
scrum-project #17
Use-case-06: External website links
Use-Cases
Scenario 3 - John wants to be link customer bu...
Public Chat
scrum-project #32
As a Project Manager, I want to create a group message so that my team can communicate in a shared conversation. (AC-04-01) (AC-04-02)
Medium
Public Chat
scrum-project #33
As a Project Manager, I want to add specific users to a group message so that only relevant team members are included. (AC-04-03) (AC-04-04)
Medium
Public Chat
scrum-project #34
As a Project Manager, I want to name a group message so that team members can easily identify its purpose. (AC-04-05)
Medium
Public Chat
scrum-project #30
As a user, I would like pasted/typed out URLs to automatically be links (AC-06-03)
Medium
Public Chat
scrum-project #38
Change message text box from input to div contentaeditable to allow links in chat
Medium
Public Chat
scrum-project #43
Public chat and private chat are visually distinct (separate views or panels) (F1.7)
Medium
Public Chat
scrum-project #31
As a user, I would like a clickable clipboard button in the message box so I can send links (AC-06-01) (AC-06-02)
Medium
Public Chat
scrum-project #44
Real-time typing status indicator for public and private chat (F1.8)
Medium
Public Chat
scrum-project #8
Persona 1 Carl - Team Member User
Personas
Public Chat
0 / 1
0%
scrum-project #10
Scenario 1 Carl - Carl wants to notify team members about an upcoming deadline.
Scenarios
Persona 1 Carl - Team Member User #8
Public Chat
0 / 3
0%
scrum-project #15
Persona 3 John - Product Support User
Personas
Public Chat
0 / 1
0%
scrum-project #21
Scenario 3 - John wants to be link customer bug tickets to the developers
Scenarios
Persona 3 John - Product Support User #15
Public Chat
0 / 1
0%
scrum-project #16
Persona 4 Greg - Project Manager
Personas
Public Chat
0 / 1
0%
scrum-project #20
Scenario 4 Greg - Wants to create a project team message group
Scenarios
Persona 4 Greg - Project Manager #16
Public Chat
0 / 1
0%



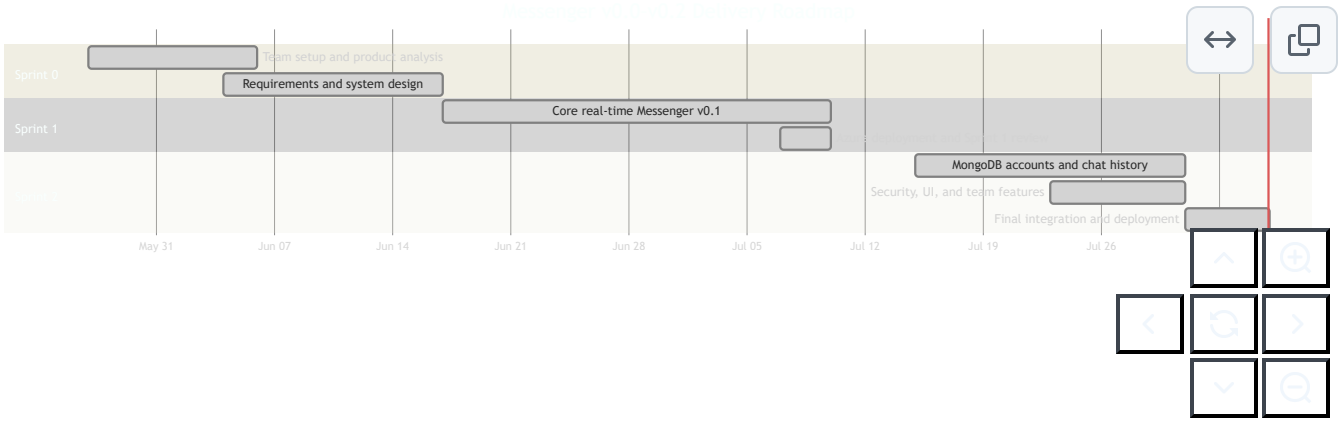
Sprint 2 Board




Scrum team planning
View 1
View 2
New view
Filter by keyword or by field
Todo 3
This item hasn't been started
In Progress 7
This is actively being worked on
Done 41
This has been completed
System Analysis 9
Used for Persona and scenario labels
Public Chat 25
Use-Case-01: Alert Team
Use-Case-02: Receive Read Receipts
Use-Case-04: Create Group Message
As a Project Manager, I want to create a group message so that my team can communicate in a shared conversation. (AC-04-01) (AC-04-02)
As a Project Manager, I want to add specific users to a group message so that only relevant team members are included. (AC-04-03) (AC-04-04)
As a Project Manager, I want to name a group message so that team members can easily identify its purpose. (AC-04-05)
Use-case-06: External website links
As a user, I would like pasted/typed out URLs to automatically be links (AC-06-03)
As a user, I would like a clickable clipboard button in the message box so I can send links (AC-06-01) (AC-06-02)
Change message text box from input to div contentaeditable to allow links in chat
Public chat and private chat are visually distinct (separate views or panels) (F1.7)
Real-time typing status indicator for public and private chat (F1.8)
Use-Case-05: Join Chat
Use-Case-06: Register Account
Use-Case-07: chat history
Use-Case-08: Logout
Use-Case-09: Account Management
Use-Case-10: Viewing accounts
Link conversion works with public chat but not private
Persona 1 Carl - Team Member User
Scenario 1 Carl - Carl wants to notify team members about an upcoming deadline.
Persona 3 John - Product Support User
Persona 4 Greg - Project Manager
Scenario 4 Greg - Wants to create a project team message group
Scenario 3 - John wants to be link customer bug tickets to the developers

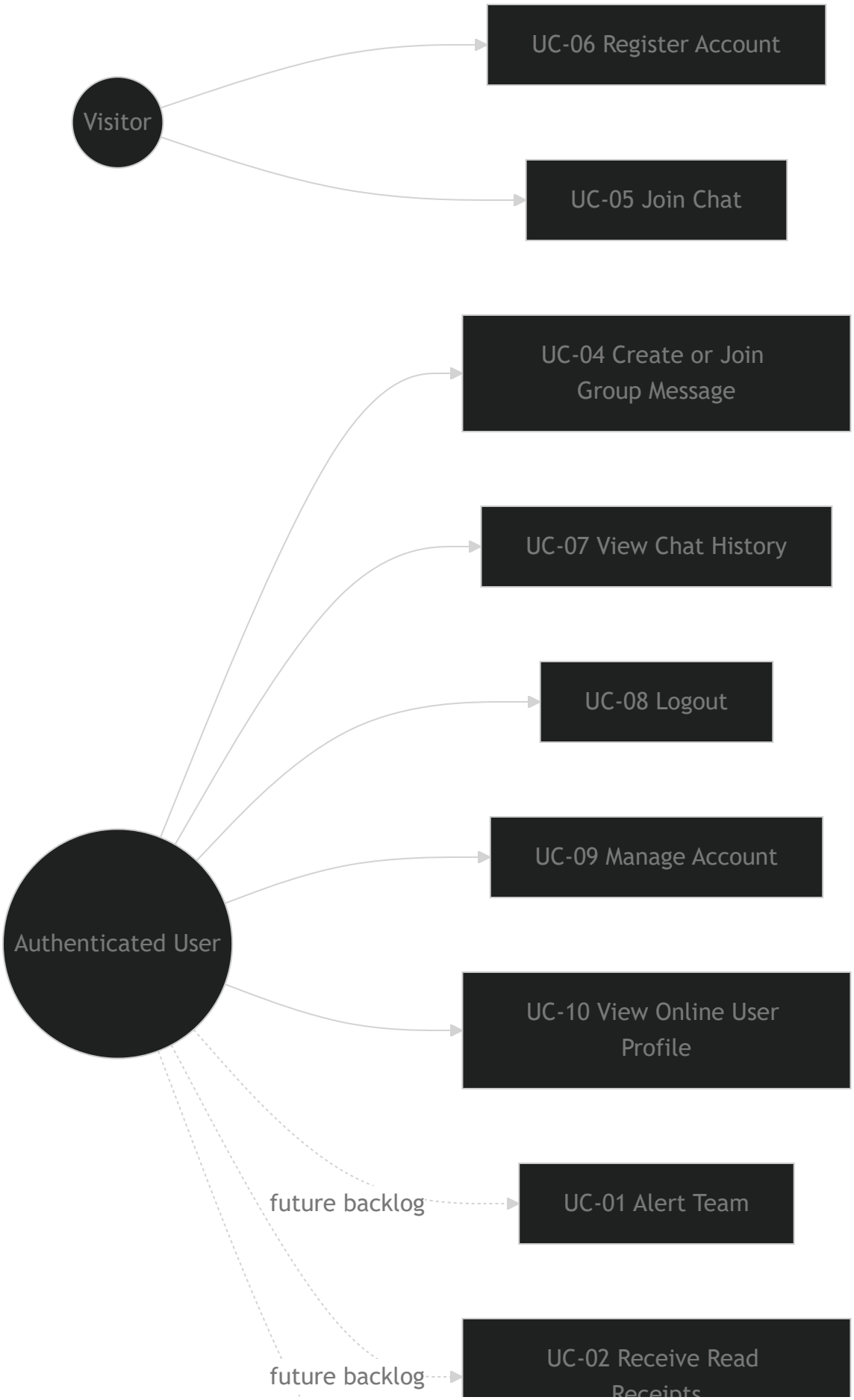


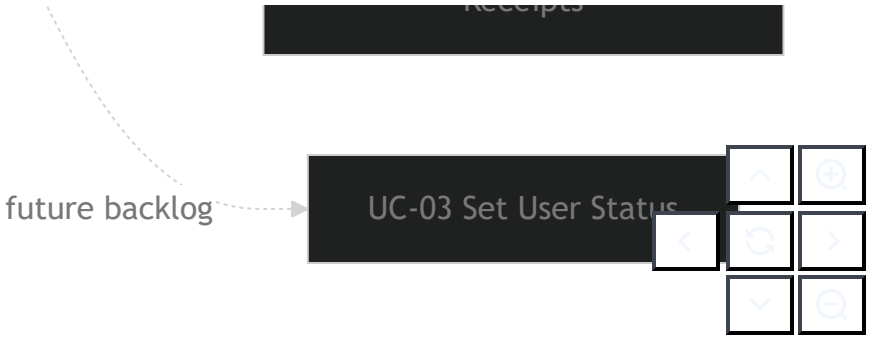
Delivery Roadmap



Use Cases

The following diagram and table distinguish implemented v0.2 behavior from future product-backlog ideas. This prevents the final design from implying that planned Sprint 0 features are already present.


Visitor
UC-06 Register Account
UC-05 Join Chat
Authenticated User
UC-04 Create or Join Group Message
UC-07 View Chat History
UC-08 Logout
UC-09 Manage Account
UC-10 View Online User Profile
future backlog
UC-01 Alert Team
future backlog
UC-02 Receive Read Receipts



UC ID	Verb-noun use case	Primary actor	Final description	Status
UC-01	Alert Team	Authenticated user	Mention a team or user and notify the affected members.	Designed; future backlog
UC-02	Receive Read Receipts	Authenticated user	See whether recipients have read a group message.	Designed; future backlog
UC-03	Set User Status	Authenticated user	Select and publish an availability status.	Designed; future backlog
UC-04	Create or Join Group Message	Authenticated user	Enter a group name, join its Socket.IO room, and exchange messages with members of that room.	Implemented with name-based membership
UC-05	Join Chat	Registered user	Submit a username/password; enter chat only when MongoDB lookup and bcrypt comparison succeed.	Implemented
UC-06	Register Account	Visitor	Create an account with a unique username, password, email, and full name.	Implemented
UC-07	View Chat History	Authenticated user	Automatically view the most recent persisted public messages after login.	Implemented
UC-08	Logout	Authenticated user	End the authenticated socket state, leave the active group, and return to login.	Implemented

UC ID	Verb-noun use case	Primary actor	Final description	Status
UC-09	Manage Account	Authenticated user	Verify the current password and update password, email, and/or full name.	Implemented
UC-10	View Online User Profile	Authenticated user	Select an online user and view the user's projected username, full name, and email.	Implemented

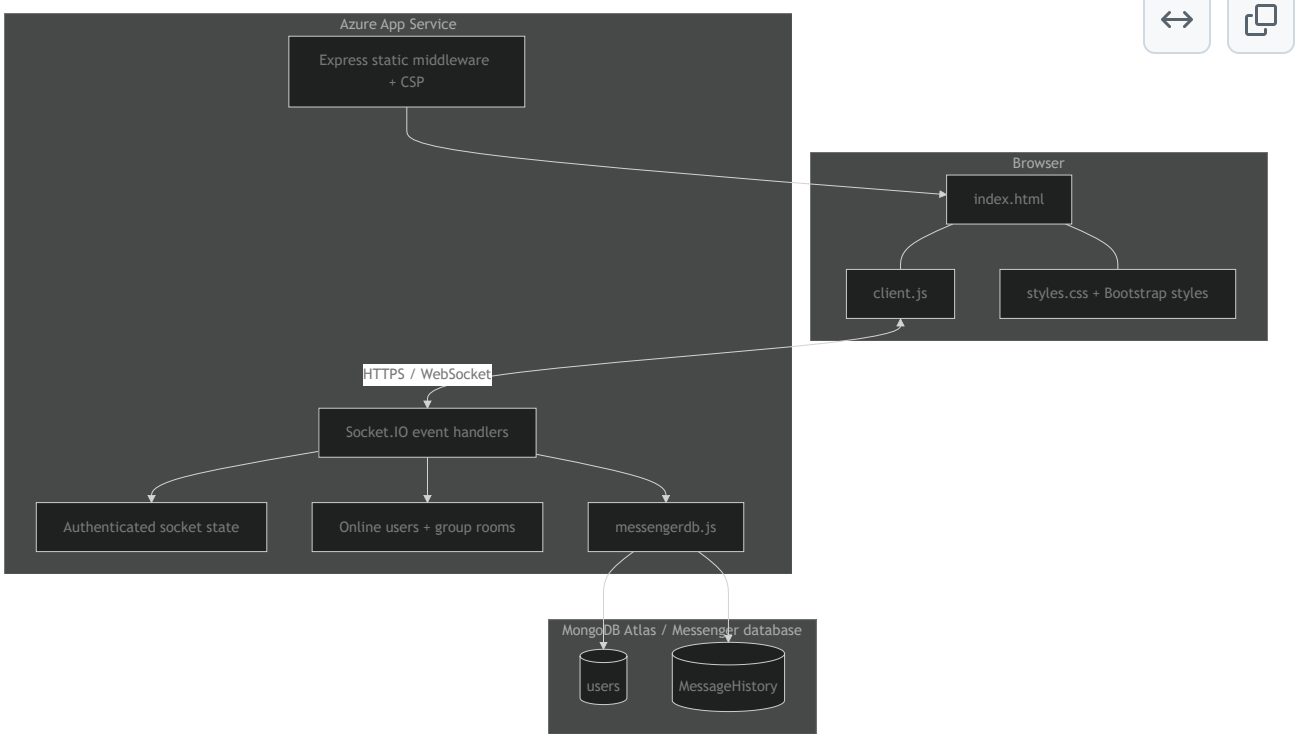
External link insertion is the implemented F1.9 feature and retains its original Sprint 1 sequence diagram under System Design.

System Design

Architecture

Messenger v0.2 follows a layered client-server design:

1. **Presentation layer:** `ui/index.html`, `ui/styles.css`, and `ui/client.js` implement the single-page interface, local validation, view switching, and Socket.IO client events.
2. **Application layer:** `server.js` serves static assets, sets CSP, manages socket authentication and online/group state, validates event data, and coordinates real-time behavior.
3. **Data-access layer:** `Database Files/messengerdb.js` owns MongoDB queries, bcrypt operations, data-layer validation, history persistence, profile updates, and soft deletion.
4. **Data layer:** MongoDB Atlas stores `users` and `MessageHistory`; in-memory maps hold only current socket and group membership state.
5. **Delivery layer:** GitHub Actions builds from `main` or `StagingEnv`, transfers a deployment artifact, authenticates to Azure through OIDC, and deploys to the corresponding App Service.



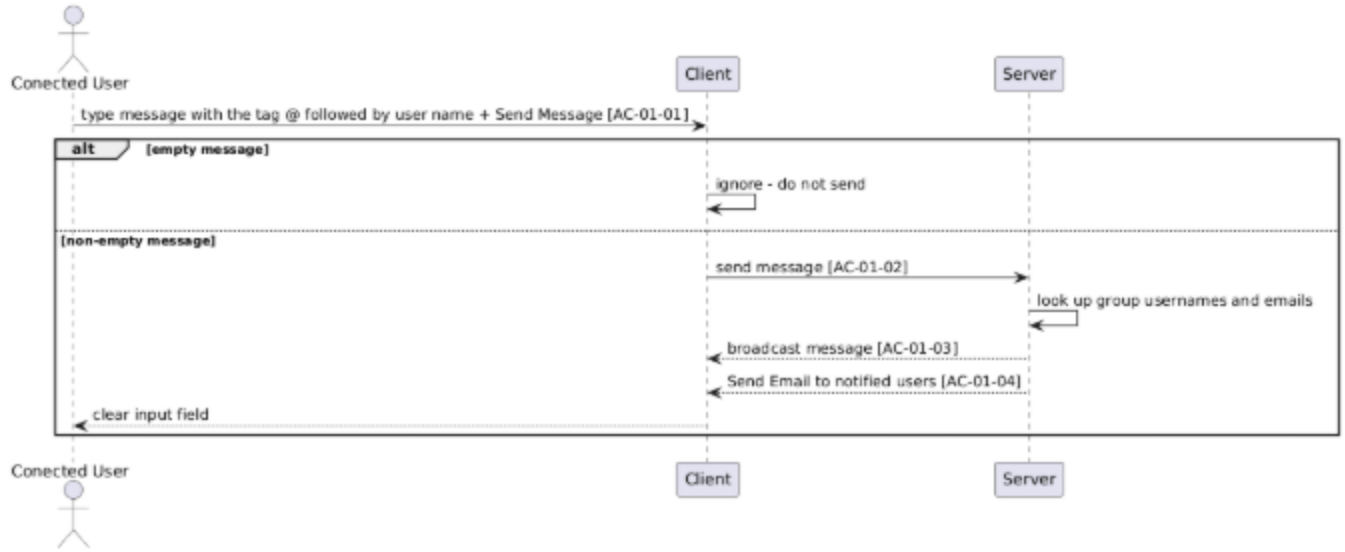
Use-Case Realization

Use case/feature	Client interaction	Server events/modules	Data or state
UC-01 Alert Team	Planned mention UI	Not present in v0.2	Future backlog
UC-02 Read Receipts	Planned read indicator	Not present in v0.2	Future backlog
UC-03 Set Status	Planned availability selector	Not present in v0.2	Future backlog

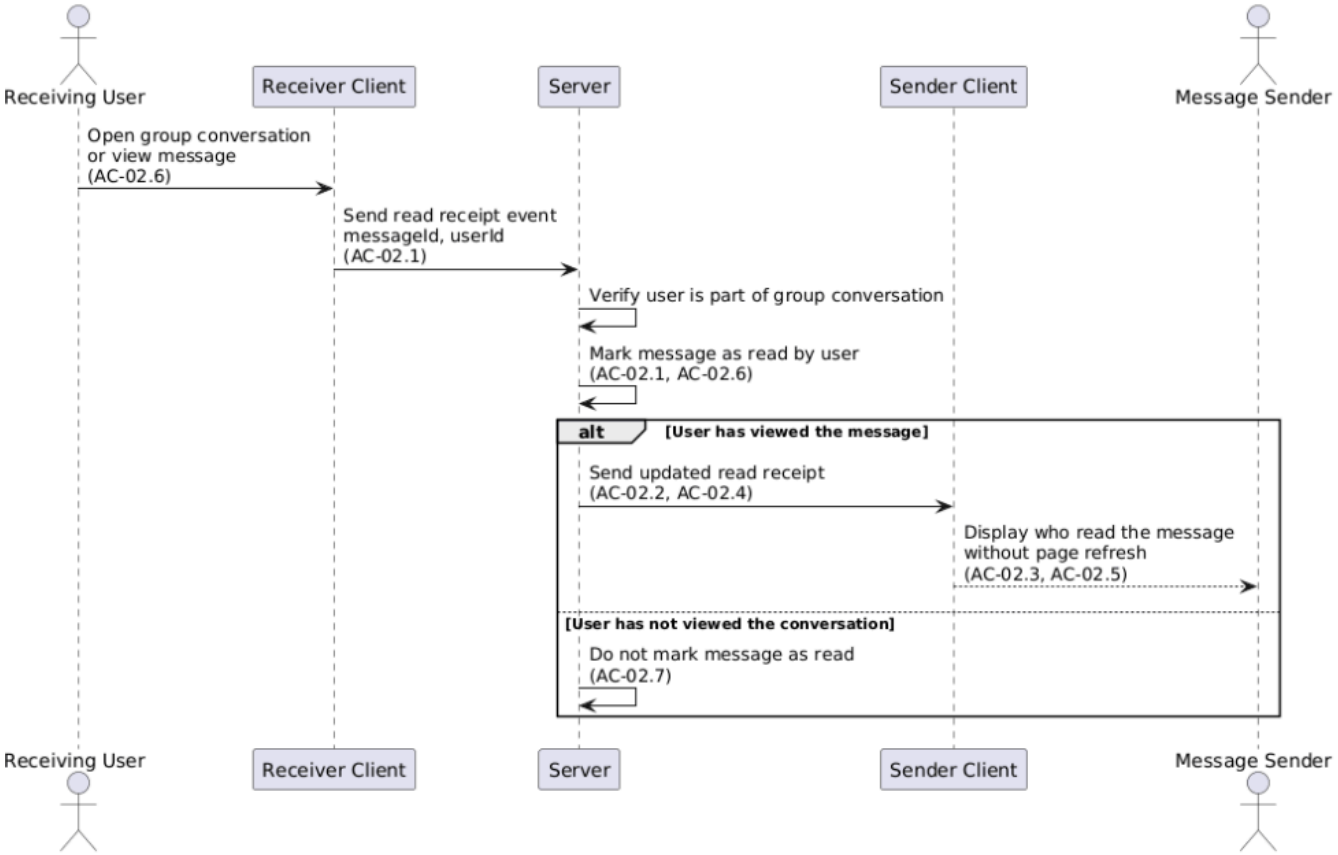
Use case/feature	Client interaction	Server events/modules	Data or state
UC-04 Group Message	Group name, join/leave, group composer	`join-group`, `leave-group`, `group-message`; Socket.IO rooms	In-memory `groups` map; not persistent
UC-05 Join Chat	Login form	`join-chat`, `authorizeUser()`, `messengerdb.find()`	`users`; authenticated socket flag
UC-06 Register	Registration form and local validation	`register`, `messengerdb.register()`	New `users` document with bcrypt hash
UC-07 Chat History	Public response pane	`loadChatHistory()`, `chat_history`	Latest 100 `MessageHistory` documents
UC-08 Logout	Logout button and view reset	`logout`, group cleanup, user-list broadcast	Auth flag and in-memory presence removed
UC-09 Manage Account	Account Management form	`edit-profile`, `messengerdb.editProfile()`	Selected fields updated in `users`
UC-10 View Profile	Select an online username	`get-user-profile`, `getPublicProfile()`	Projected public fields from `users`
F1.9 External Links	Link dialog or automatic URL conversion	Existing public/private message events	HTTP(S)-only anchors in message HTML
F2.7 Delete Message	Sender selects the `x` button	`delete-message`, `messengerdb.deleteMsg()`	Owner-checked soft delete in `MessageHistory`

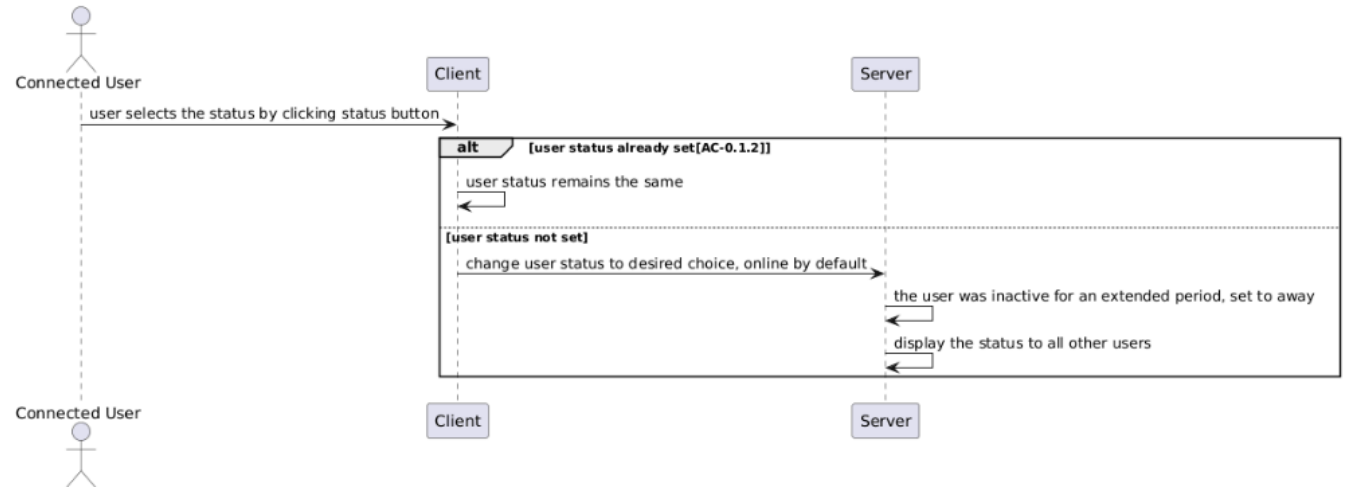
Sequence Diagrams

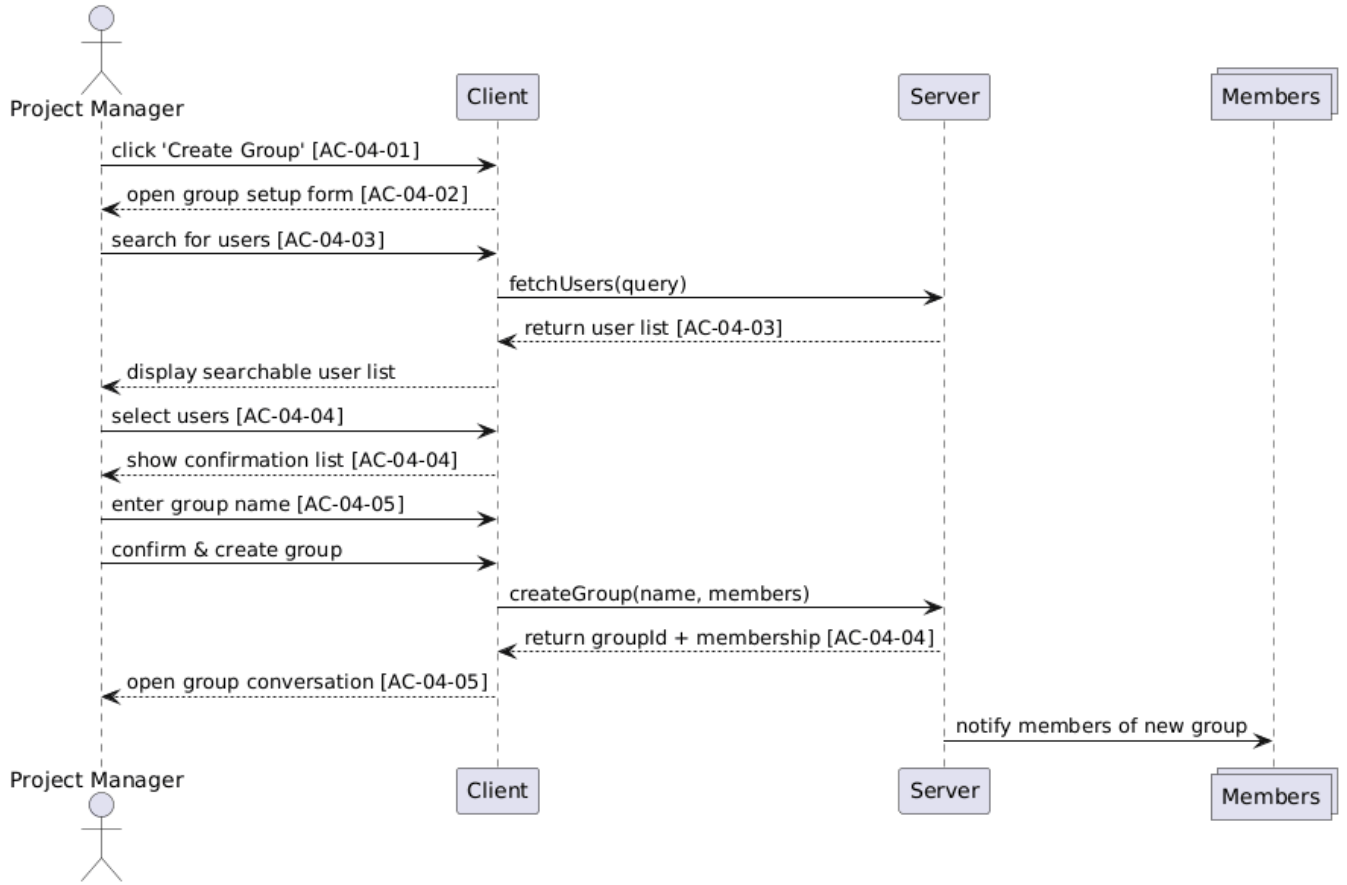
The Sprint 0 diagrams below document the planned flows for UC-01 through UC-04. The realization table marks the first three as future backlog so the design remains consistent with v0.2.



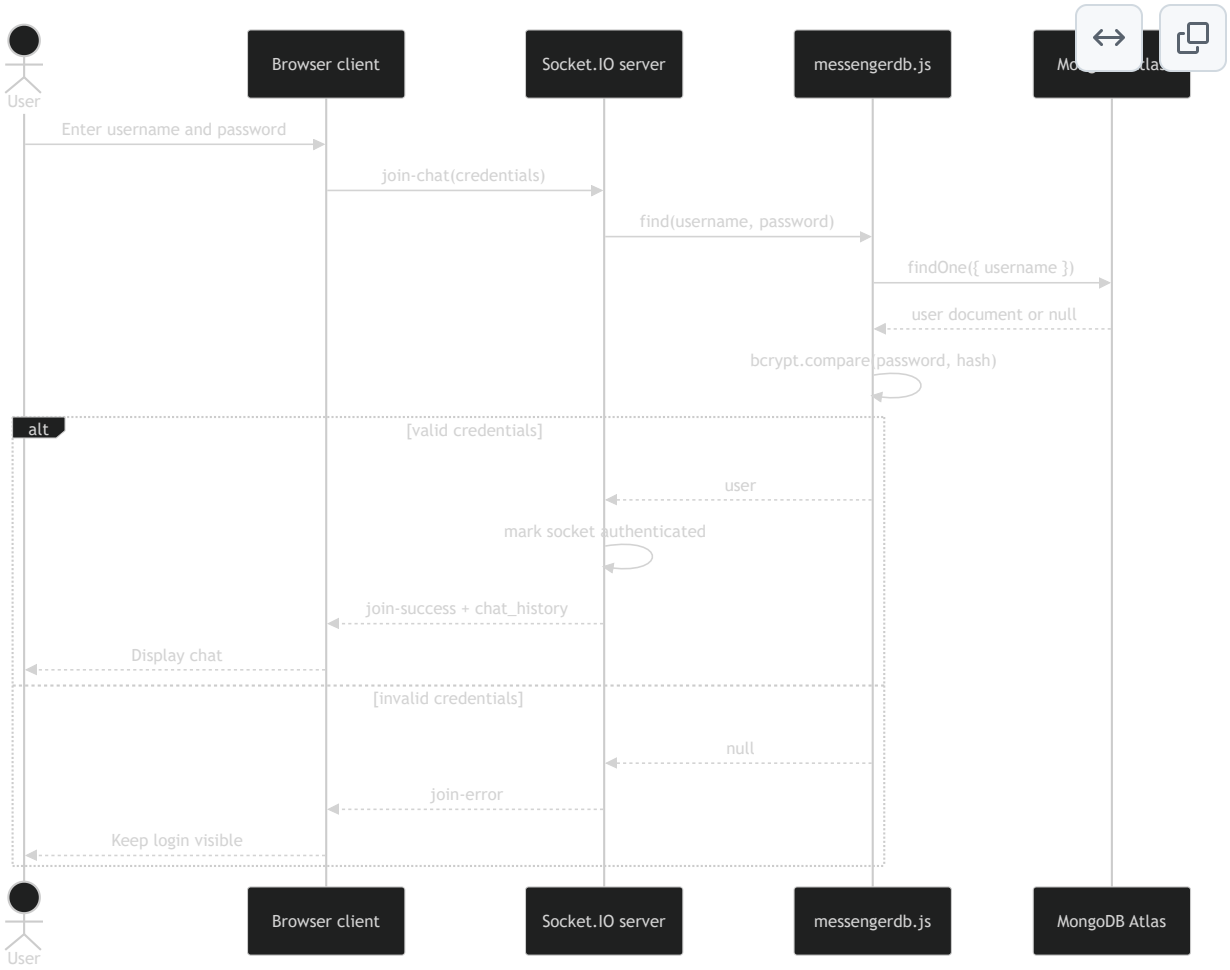
Use-Case-02: Receive Read Receipts

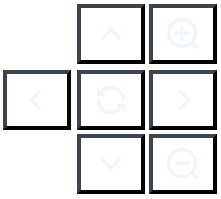




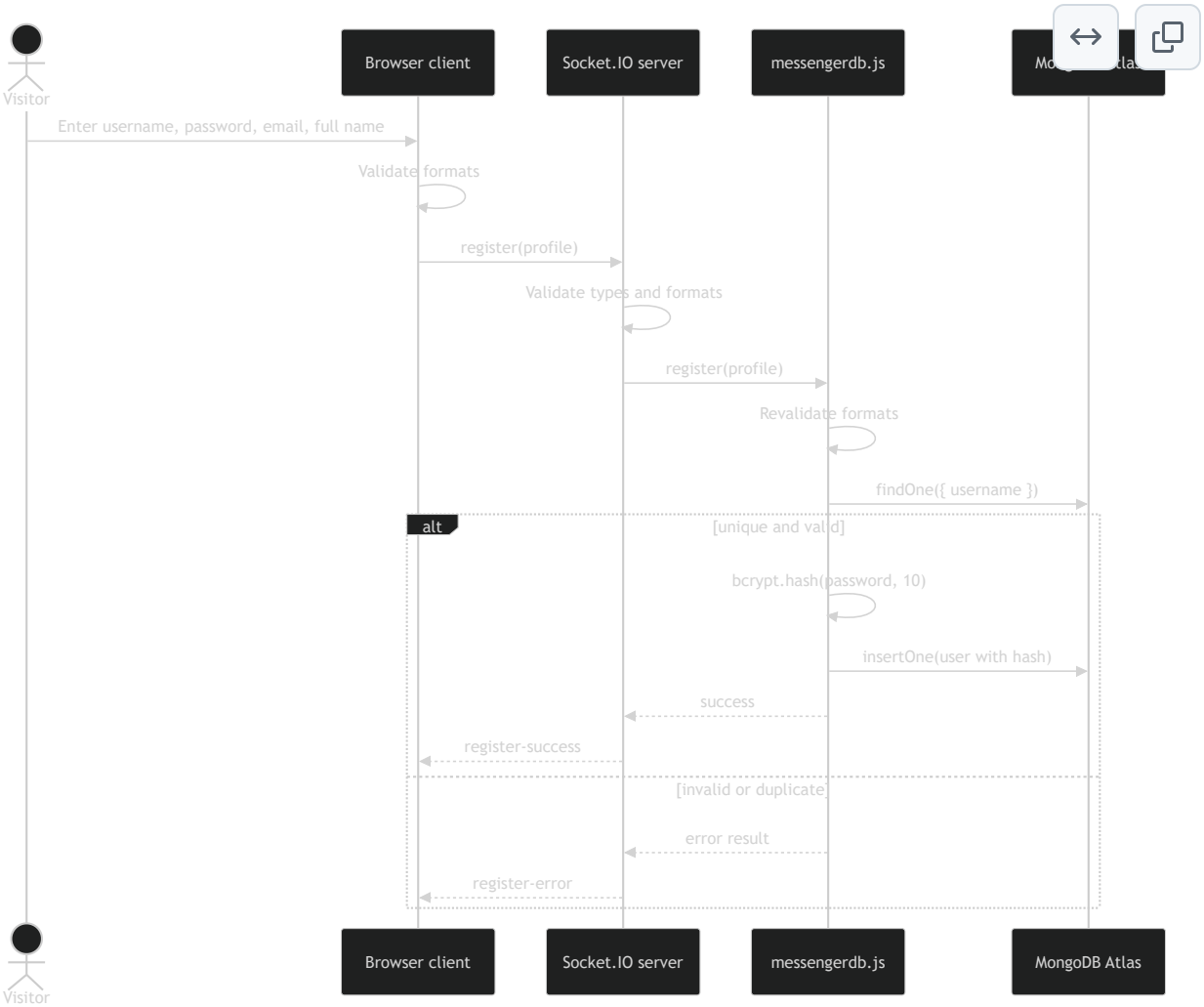


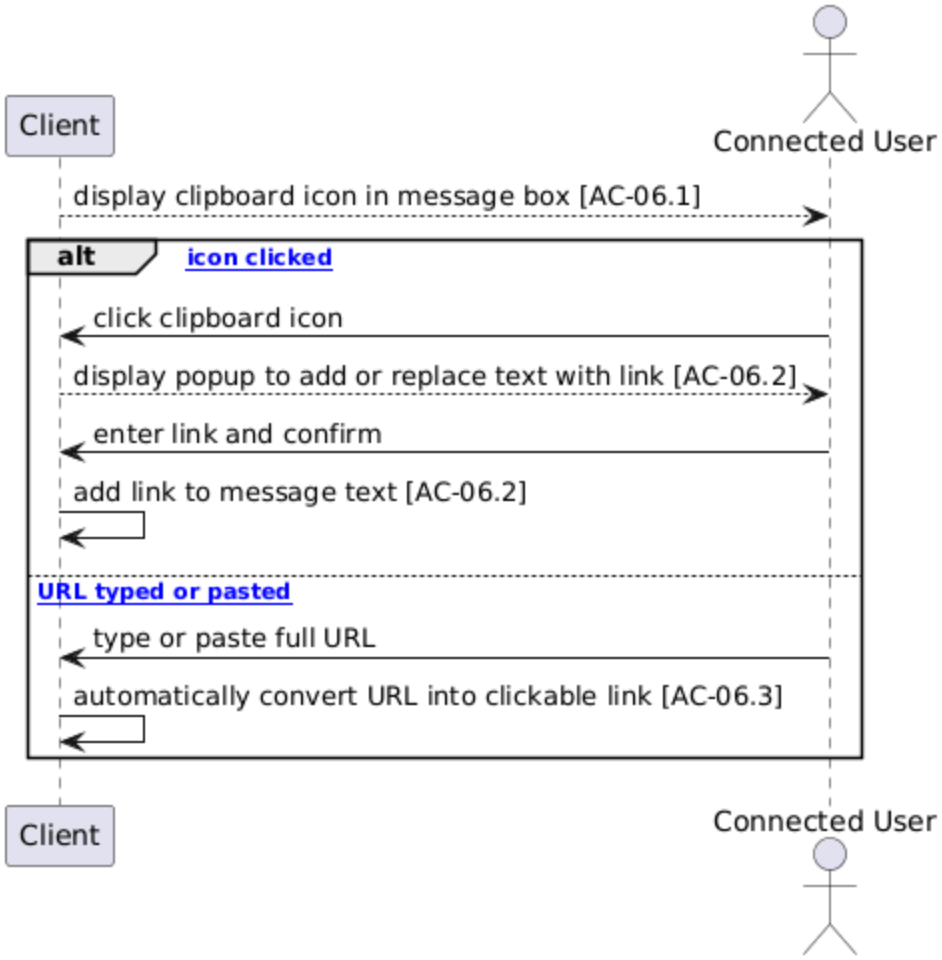
UC-05 authentication is realized by the following final flow:

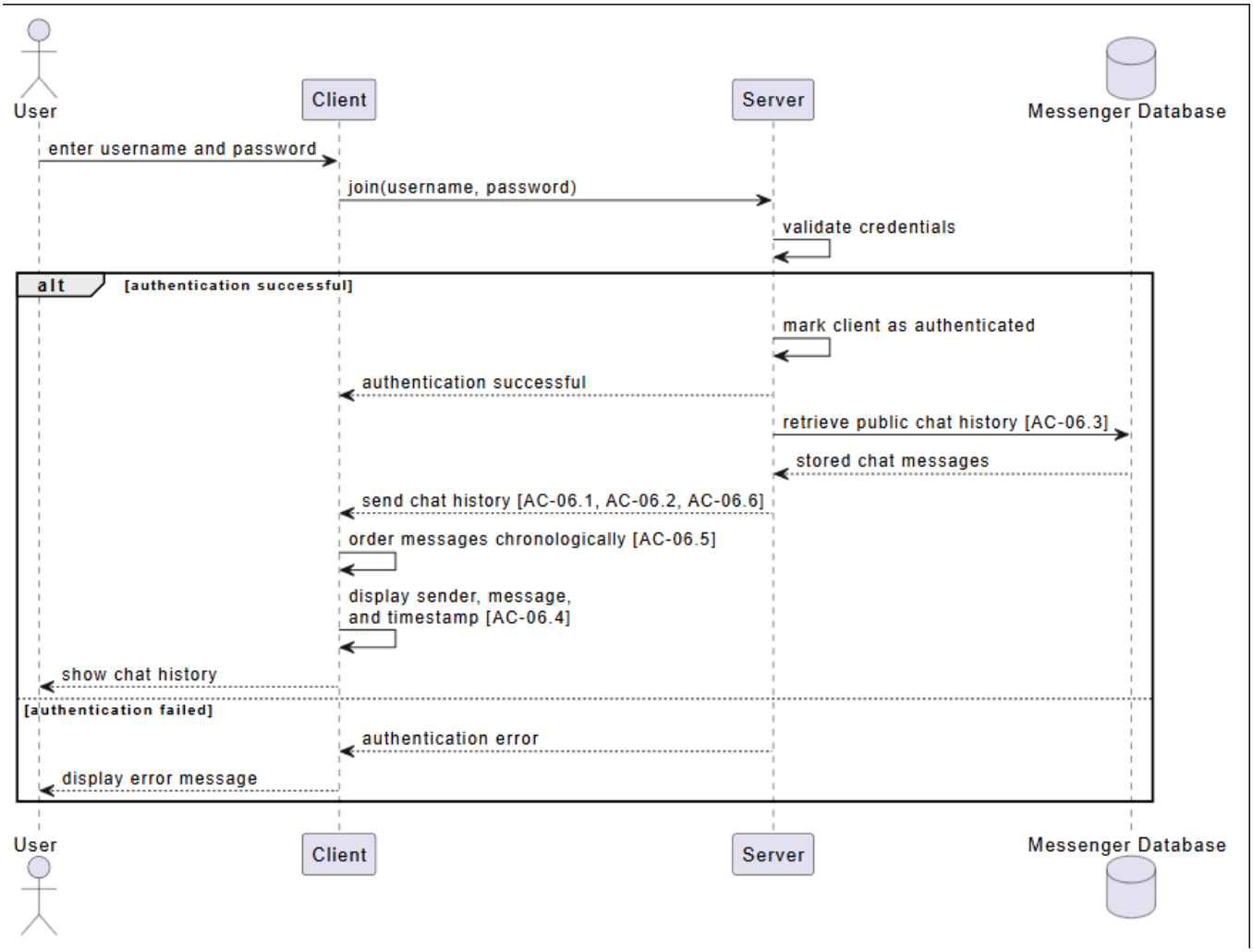


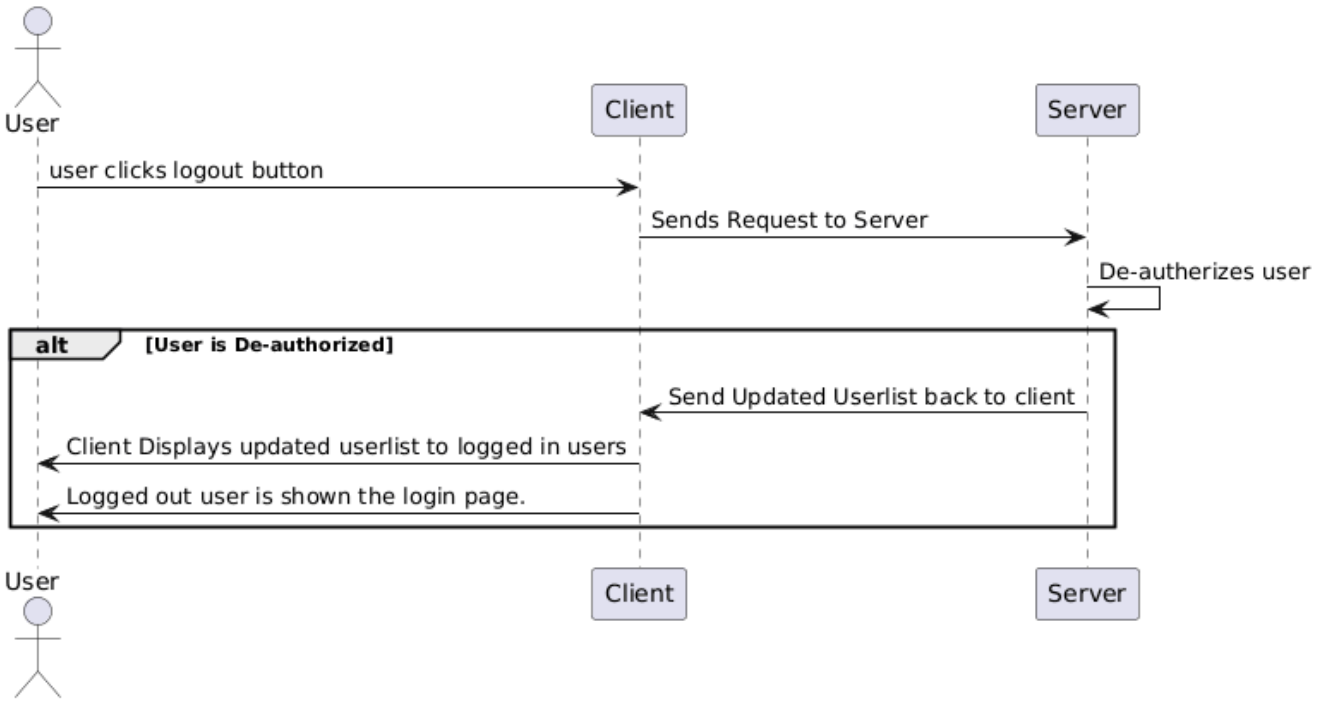


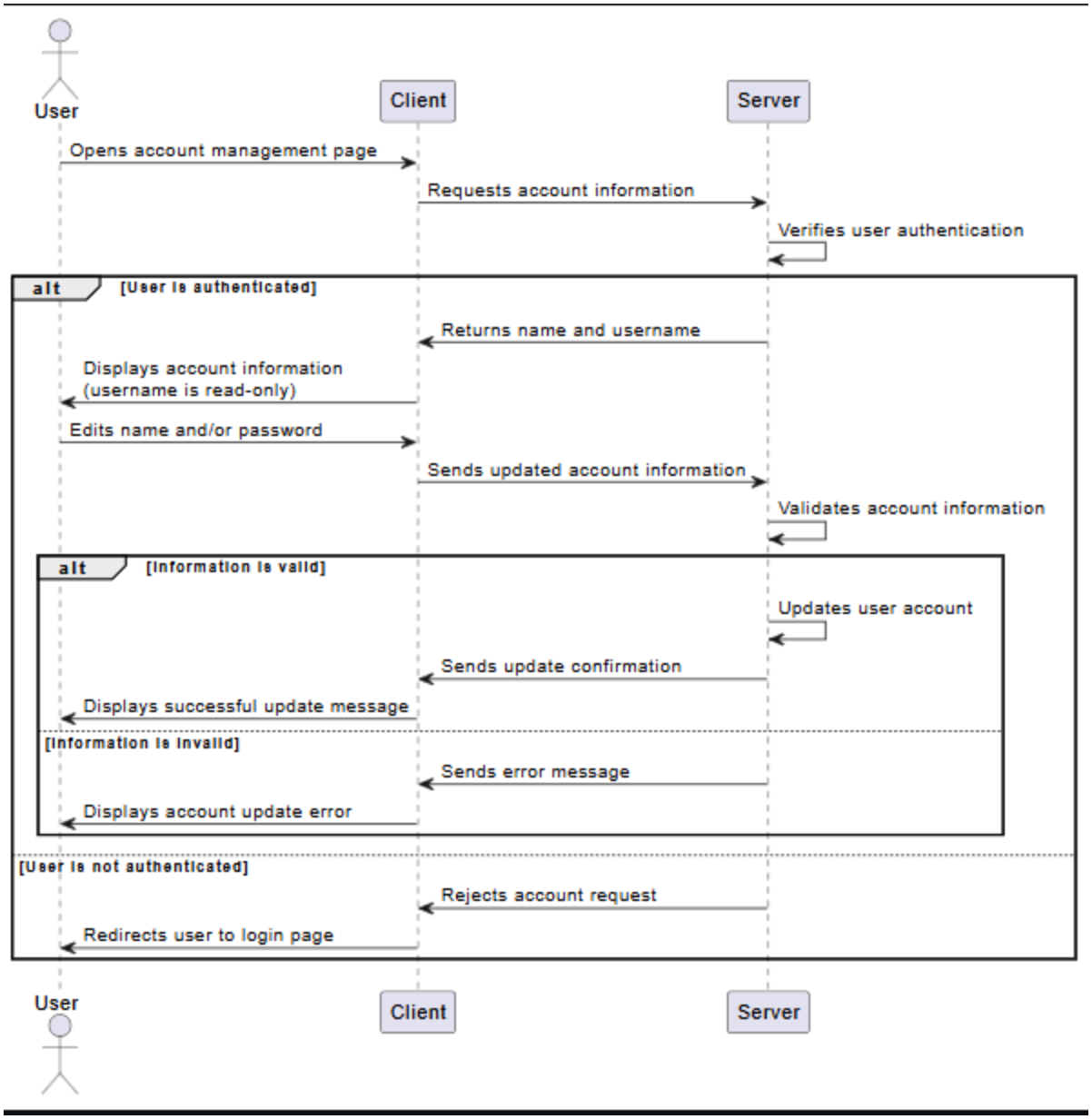
UC-06 registration uses client, server, and data-layer validation:

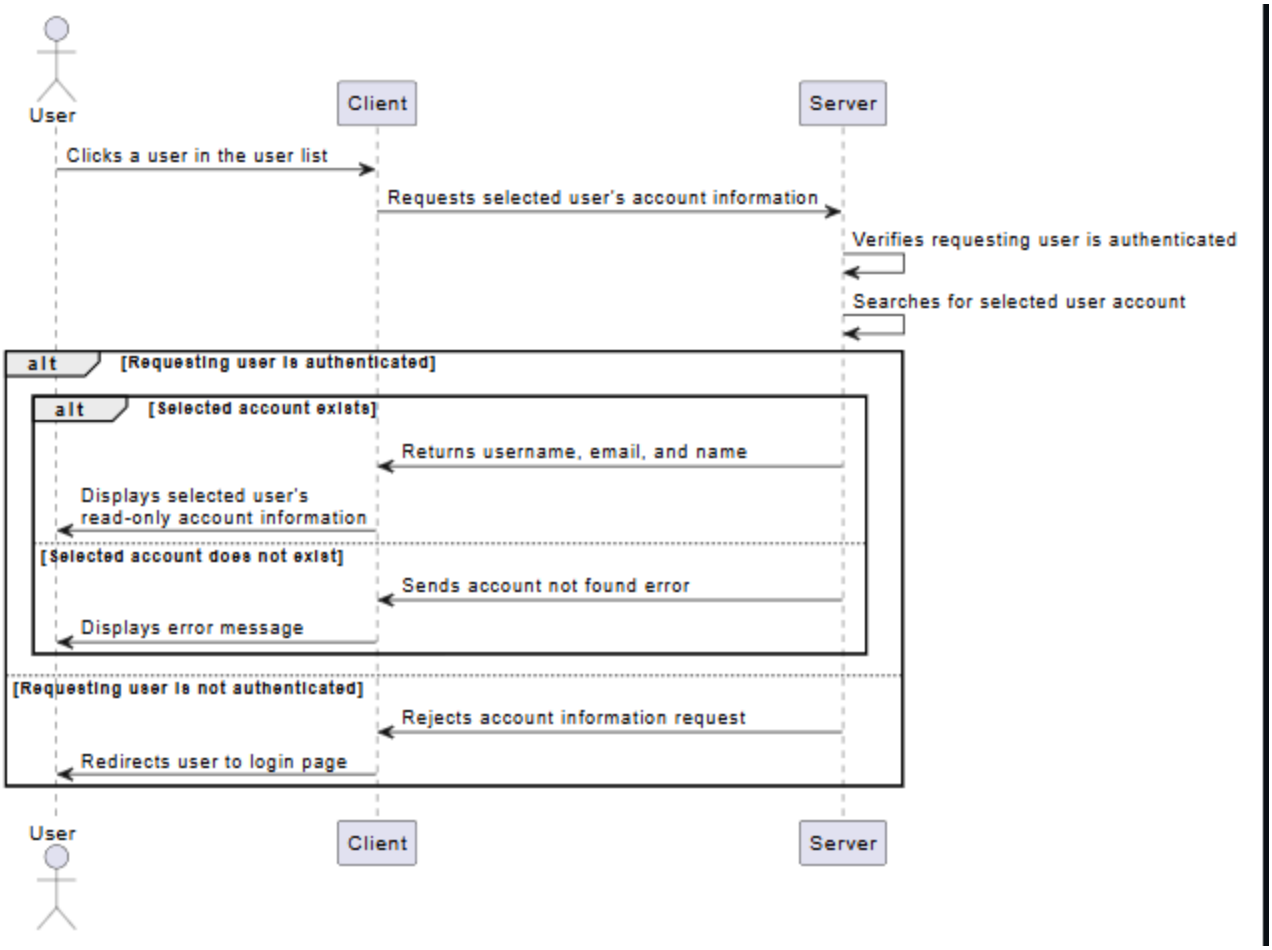

Visitor
Browser client
Socket.IO server
messengerdb.js
MongoDB Atlas
Enter username, password, email, full name
Validate formats
register(profile)
Validate types and formats
register(profile)
Revalidate formats
findOne({ username })
alt
[unique and valid]
bcrypt.hash(password, 10)
insertOne(user with hash)
success
register-success
[invalid or duplicate]
error result
register-error
Visitor
Browser client
Socket.IO server
messengerdb.js
MongoDB Atlas


Client
Connected User
display clipboard icon in message box [AC-06.1]
alt
icon clicked
click clipboard icon
display popup to add or replace text with link [AC-06.2]
enter link and confirm
add link to message text [AC-06.2]
URL typed or pasted
type or paste full URL
automatically convert URL into clickable link [AC-06.3]
Client
Connected User


User
Client
Server
Messenger Database
enter username and password
join(username, password)
validate credentials
alt
[authentication successful]
mark client as authenticated
authentication successful
retrieve public chat history [AC-06.3]
stored chat messages
send chat history [AC-06.1, AC-06.2, AC-06.6]
order messages chronologically [AC-06.5]
display sender, message,
and timestamp [AC-06.4]
show chat history
[authentication failed]
authentication error
display error message
User
Client
Server
Messenger Database


User
Client
Server
user clicks logout button
Sends Request to Server
De-autherizes user
alt
[User is De-authorized]
Send Updated Userlist back to client
Client Displays updated userlist to logged in users
Logged out user is shown the login page.
User
Client
Server

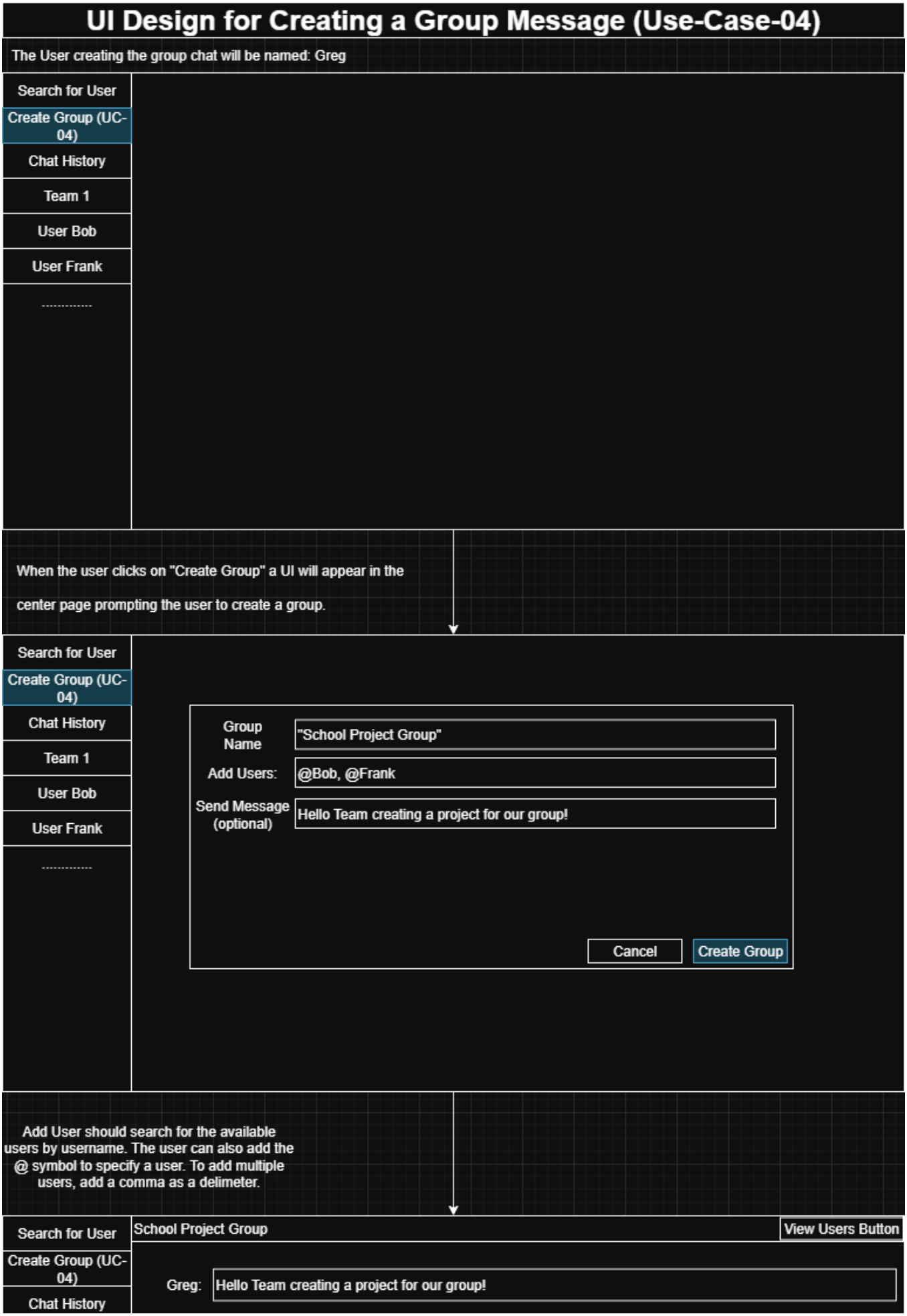

User
Client
Server
Opens account management page
Requests account information
Verifies user authentication
alt
[User is authenticated]
Returns name and username
Displays account information
(username is read-only)
Edits name and/or password
Sends updated account information
Validates account information
alt
[Information is valid]
Updates user account
Sends update confirmation
Displays successful update message
[Information is invalid]
Sends error message
Displays account update error
[User is not authenticated]
Rejects account request
Redirects user to login page
User
Client
Server



User Interface

The v0.2 client is a single-page application with three unauthenticated/authenticated states: login, registration, and chat. A successful login hides the authentication views and displays a sidebar plus separate public, private, and group panels. Account Management temporarily replaces the chat view. Selecting an online username opens a read-only profile dialog.

The following Sprint 0 mockups established the interaction concepts later implemented in the application:

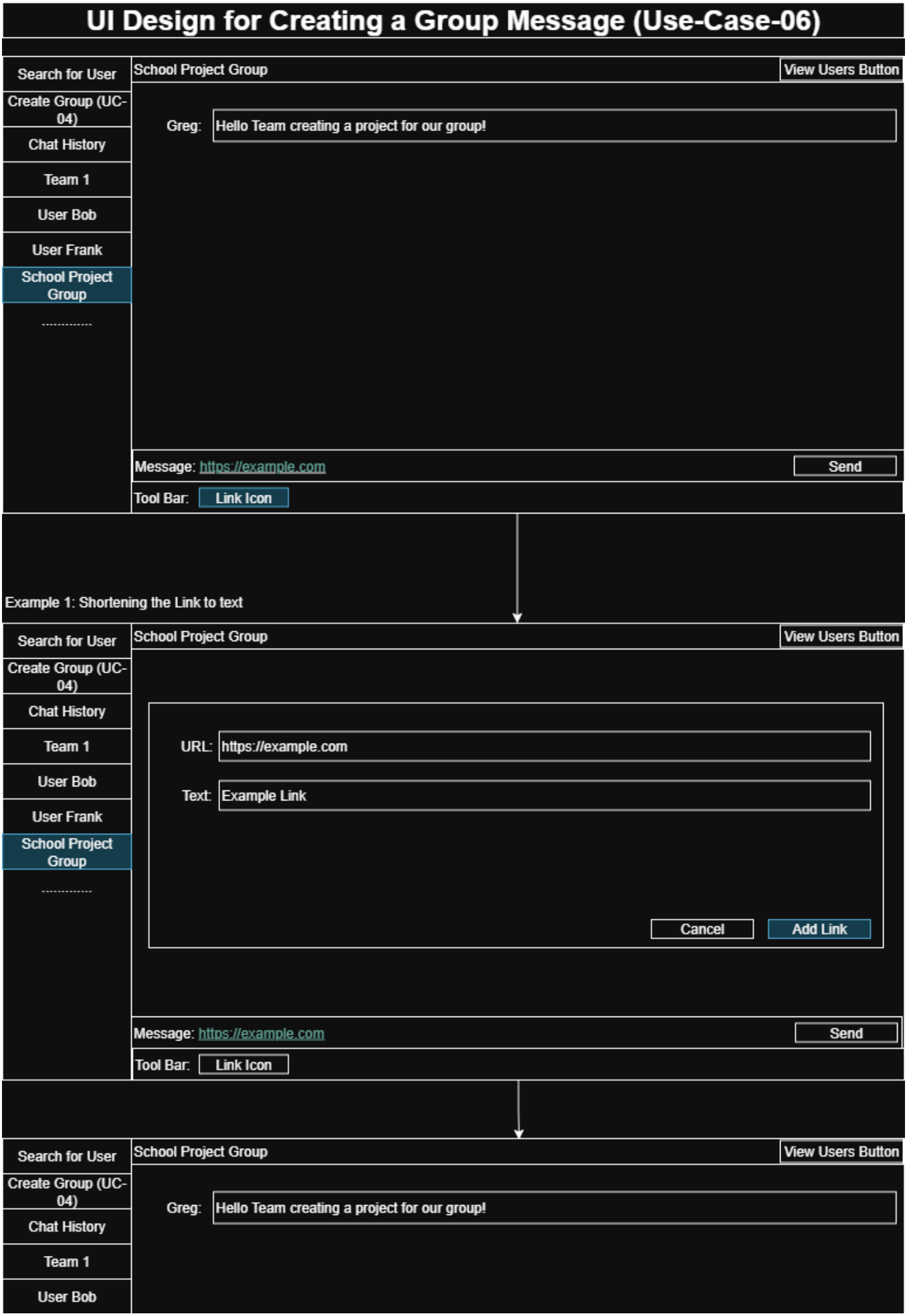

UI Design for Creating a Group Message (Use-Case-04)
The User creating the group chat will be named: Greg
Search for User
Create Group (UC-04)
Chat History
Team 1
User Bob
User Frank

When the user clicks on "Create Group" a UI will appear in the center page prompting the user to create a group.
Search for User
Create Group (UC-04)
Chat History
Team 1
User Bob
User Frank

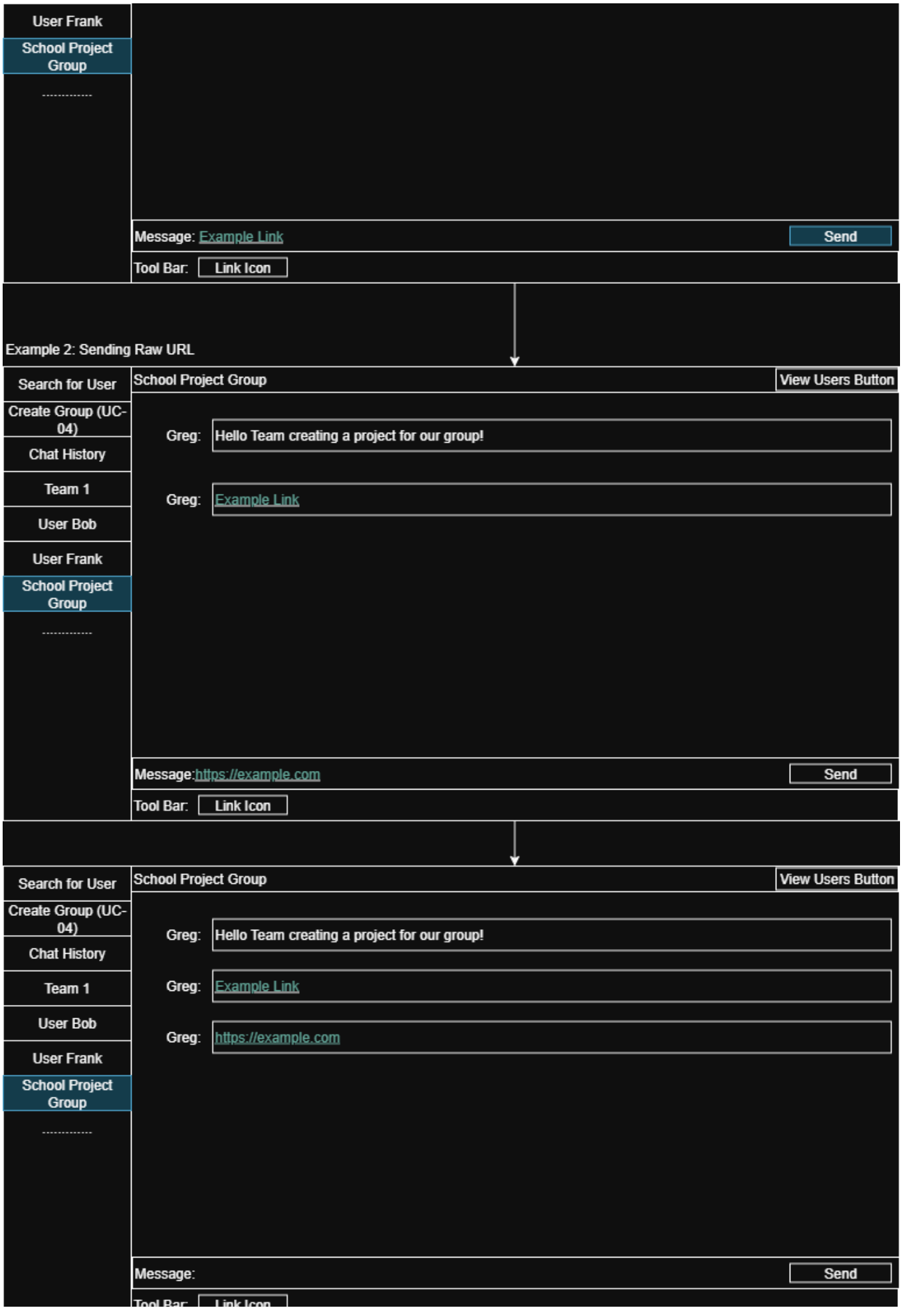
Group Name
"School Project Group"
Add Users:
@Bob, @Frank
Send Message (optional)
Hello Team creating a project for our group!
Cancel
Create Group
Add User should search for the available users by username. The user can also add the @ symbol to specify a user. To add multiple users, add a comma as a delimeter.
Search for User
Create Group (UC-04)
Chat History
School Project Group
View Users Button
Greg:
Hello Team creating a project for our group!



The final v0.2 group implementation uses a shared group name instead of the mockup's invitation list. Every intended member enters the same group name and selects **Join Group**.


UI Design for Creating a Group Message (Use-Case-06)
Search for User
Create Group (UC-04)
Chat History
Team 1
User Bob
User Frank
School Project Group

School Project Group
View Users Button
Greg:
Hello Team creating a project for our group!
Message: https://example.com
Send
Tool Bar:
Link Icon
Example 1: Shortening the Link to text
URL:
https://example.com
Text:
Example Link
Cancel
Add Link


User Frank
School Project Group

Message: Example Link
Send
Tool Bar:
Link Icon
Example 2: Sending Raw URL
Search for User
Create Group (UC-04)
Chat History
Team 1
User Bob
User Frank
School Project Group

School Project Group
View Users Button
Greg: Hello Team creating a project for our group!
Greg: Example Link
Message:https://example.com
Send
Tool Bar:
Link Icon
Search for User
Create Group (UC-04)
Chat History
Team 1
User Bob
User Frank
School Project Group

School Project Group
View Users Button
Greg: Hello Team creating a project for our group!
Greg: Example Link
Greg: https://example.com
Message:
Send
Tool Bar:
Link Icon



The final link interface supports both the popup shown in the mockup and automatic HTTP(S) URL conversion in public and private composers.

Database

Database name: `Messenger`. The data-access module uses two collections.

`users`

Field	Type	Meaning
`_id`	ObjectId	MongoDB document identifier
`username`	String	Login identifier, 3–20 letters/numbers/underscores
`password`	String	bcrypt hash generated with cost factor 10
`email`	String	Validated profile email
`fullName`	String	Display/contact name

```
{
  "_id": "ObjectId('...')",
  "username": "student_1",
  "password": "$2b$10$<bcrypt-hash>",
  "email": "student1@example.edu",
  "fullName": "Student One"
}
```

`MessageHistory`

Field	Type	Meaning
`_id`	ObjectId	MongoDB document identifier
`sender`	String	Username captured from the authenticated socket
`message`	String	Public message HTML/text
`timestamp`	Number	Milliseconds since the Unix epoch
`deleted`	Boolean	Whether the sender soft-deleted the message

```
{
  "_id": "ObjectId('...')",
  "sender": "student_1",
  "message": "Project meeting at 3:00 PM",
  "timestamp": 1785524400000,
  "deleted": false
}
```

There is no stored foreign key: `MessageHistory.sender` logically references `users.username`. The current source creates no custom indexes, so only MongoDB's default `_id` indexes are guaranteed by code. Production hardening should add a unique index on `users.username` to make duplicate-account prevention atomic and a descending index on `MessageHistory.timestamp` to support the history query. Private and group messages are currently transient.

Security (SSDLC)

Security is applied at the client, Socket.IO server, data-access module, deployment configuration, and database boundary. The Sprint 2 implementation introduced hashed passwords, repeated input validation, authenticated socket checks, field-limited updates/projections, a CSP response header, HTTPS deployment, and environment-based secret storage. The security review also records residual risk rather than treating the current controls as complete.

Security Requirements

ID	Requirement	Implementation/status
SR-1	Production client/server traffic shall use HTTPS.	The documented Azure endpoint uses HTTPS.
SR-2	Passwords shall be hashed before storage and never logged.	`bcrypt.hash(..., 10)` stores hashes; login uses `bcrypt.compare()`; debug logs omit passwords.
SR-3	Registration and profile inputs shall be validated at every layer that consumes them.	Username/password/email/full-name checks exist in the browser, server, and database module; profile changes require current-password verification.

ID	Requirement	Implementation/status
SR-4	MongoDB selectors shall be built from validated primitive values, not client-provided query objects.	Login rejects non-string credentials; registration uses regex checks; profile updates use a server-derived username and an allowlisted update object.
SR-5	Protected chat/profile operations shall require an authenticated socket.	`authorizeUser()` guards message, group, typing, profile, edit, and logout flows; broadcasts target authenticated sockets.
SR-6	The browser shall receive a restrictive Content Security Policy.	Express sets `default-src 'self'` and explicit script, style, connection, and font source allowlists. The live response was verified on August 4, 2026.
SR-7	Database credentials shall not be committed in source.	`MONGO_URI` is loaded from `.env` locally and Azure configuration in production; `.env` is ignored.
SR-8	User-generated message markup shall be sanitized before DOM insertion.	**Open risk:** v0.2 still transports composer HTML and inserts it with `innerHTML`; DOMPurify or equivalent sanitization is not present. CSP reduces exploitability but is not a sanitizer.

Defense-in-Depth Validation

Registration

Layer	Checks
Client	Username length/pattern, password letter-and-number rule, email format, non-empty full name
Server	Payload presence and primitive types, trimmed values, repeated username/password/email formats
Data access	Repeats formats, checks for an existing username, hashes the password, inserts only named fields
Database	Stores the bcrypt hash rather than the submitted plaintext password

Profile Update

Layer	Checks
Client	Requires current password and at least one proposed change
Server	Requires authenticated socket and a string current password; derives username from authenticated state
Data access	Compares current password hash, validates new password/email, constructs an allowlisted `$set` object, and re-hashes changed passwords
Database	Updates only the authenticated user's document

Threat Model

Assets

- Password hashes and account/profile data
- Public chat history and message ownership
- Private and group message confidentiality while connected
- Authenticated socket state, online-user list, and group membership
- MongoDB connection credentials and Azure deployment configuration
- Application availability and integrity

Trust Boundaries

1. User input crossing into the browser DOM and Socket.IO client.
2. Socket.IO data crossing from an untrusted browser to the Node.js server.
3. Validated application data crossing from the server into MongoDB Atlas.
4. Deployment artifacts and OIDC credentials crossing from GitHub Actions into Azure.
5. HTTP(S) links navigating from Messenger to external websites.

STRIDE category	Threat	Current control	Residual risk/action
Spoofing	Invalid user attempts to impersonate an account.	MongoDB username lookup plus bcrypt comparison; generic login failure.	No login rate limiting or account lockout; credential stuffing remains possible.
Tampering / XSS	A message contains	CSP has no `unsafe-inline` script	Raw message HTML reaches `innerHTML`; add

STRIDE category	Threat	Current control	Residual risk/action
	executable markup.	permission and restricts script sources.	DOMPurify on send/render and server-side content validation.
Repudiation	Sender denies posting or deleting a message.	Public records retain sender, timestamp, and a deletion flag.	No immutable audit log; deletion timestamp is not recorded.
Information disclosure	Credentials or database secrets are exposed.	HTTPS URL, bcrypt hashes, `.env`/Azure configuration, password-free logs.	Previously committed Atlas credentials remain in history and must be rotated.
Information disclosure	Profile email is disclosed more broadly than intended.	Server returns only username, email, and full name and only for online users to authenticated users.	Confirm that email visibility matches the privacy requirement.
Denial of service	Flooded login, registration, or message events consume resources.	Basic empty/type/format validation.	No event rate limiter or message-size limits are implemented.
Elevation of privilege	An unauthenticated socket sends protected events.	`authorizeUser()` and authenticated-only broadcasts.	Add the same explicit authorization/type guard to `delete-message`, and emit `message-delete` only when the ownership-checked update succeeds.
NoSQL injection	Client supplies MongoDB operators instead of strings.	Primitive type checks, regex validation, server-derived usernames, and allowlisted updates.	Continue validating every new database-bound field; add tests for object/array payloads.

STRIDE category	Threat	Current control	Residual risk/action
Supply-chain compromise	A third-party browser script is altered.	jQuery, Popper, and Bootstrap scripts use Subresource Integrity and CSP source restrictions.	Pin and review dependencies regularly; reduce allowed origins when possible.

Sprint 2 Security Review Notes

The review found that a MongoDB Atlas connection string had previously been hardcoded in source. Sprint 2 moved the value into `process.env.MONGO_URI`, added `.env.example`, relied on the existing `.gitignore` rules for `.env`, and configured the variable in Azure App Service. Because a secret remains recoverable from Git history after removal, the associated Atlas database password should be rotated and the old credential disabled.

The review confirmed bcrypt hashing, database-backed password comparison, registration/profile validation at multiple layers, authenticated event checks, CSP delivery, and basic NoSQL injection guards. It also confirmed that the earlier XSS concern is not fully remediated: public and private message paths still use `innerHTML` without DOMPurify. CSP is useful defense in depth, but output sanitization remains required. The delete event also needs an explicit authorization/type check and a success-only broadcast. Rate limiting, message-size limits, automated security tests, and a unique username index are additional hardening items.

Implementation

Technology Stack

Layer	Technology
Runtime	Node.js 18 or newer locally; GitHub Actions currently selects Node.js 24.x
Web server	Express 4.22
Real-time communication	Socket.IO 4.8

Layer	Technology
Database	MongoDB Atlas through MongoDB Node.js driver 7.5
Password hashing	bcrypt 6.0, cost factor 10
Configuration	dotenv 17.4 locally; Azure App Service environment variables in production
Client	HTML5, CSS, JavaScript, Bootstrap 5, jQuery, Font Awesome
Version control	Git and private GitHub repository with branches and pull requests
Project management	GitHub Issues and GitHub Projects
Hosting	Azure App Service production and staging applications
CI/CD	GitHub Actions, artifact transfer, Azure OIDC login, `azure/webapps-deploy`

Project Layout

```
sprint2/
├── README.md
├── docs/                          diagrams and board captures
└── src/
    ├── server.js                  Express, CSP, Socket.IO application logic
    ├── Database Files/
    │   └── messengerdb.js         MongoDB and bcrypt data-access functions
    ├── ui/
    │   ├── index.html             single-page UI
    │   ├── client.js              browser events and rendering
    │   └── styles.css             application styling
    ├── .env.example               safe configuration template
    ├── package.json
    └── package-lock.json
```

Implementation by Sprint

Sprint 0

- Created personas, scenarios, use cases, acceptance criteria, sequence diagrams, wireframes, architecture, security requirements, and threat model.

- Established the organization, private source repository, public team homepage, GitHub Project, and initial application skeleton.

Sprint 1 — v0.1

- Built public and private Socket.IO messaging and the online-user list.
- Separated login, public, and private interfaces; added automatic scroll and typing status.
- Added link insertion and automatic HTTP(S) link conversion.
- Added Azure production deployment through GitHub Actions.

Sprint 2 — v0.2

- Replaced username-only entry with MongoDB-backed registration and bcrypt authentication.
- Added email/full-name profile data, password/profile editing, logout, public history loading, and owner-only soft deletion.
- Added online-user profile viewing and name-based group rooms.
- Added Bootstrap authentication views and cleaned up public/private/group chat panels.
- Added CSP, environment-based MongoDB configuration, staging deployment, and defense-in-depth validation.

Getting Started Locally

Prerequisites: Git, Node.js 18 or newer, npm, and a MongoDB Atlas connection string with access to the `Messenger` database.

```
git clone git@github.com:uc-se-sm26-team1/scrum-project.git
cd scrum-project/sprint2/src
npm install
cp .env.example .env
```

On Windows PowerShell, use `Copy-Item .env.example .env` instead of `cp` if preferred. Set the local file to:

```
MONGO_URI=mongodb+srv://<database-user>:<password>@<cluster-host>/
```

Do not commit `.env`. Then start the app:

```
npm start
```

Open http://localhost:8080. The server uses `PORT` when supplied by Azure and otherwise defaults to `8080`. Startup intentionally fails if the MongoDB connection cannot be established.

CI/CD Pipeline

The production workflow is `.github/workflows/main_team1-messengerapp.yml`. A push to `main`, or a manual dispatch, performs the following:

1. Check out the repository on Ubuntu.
2. Select Node.js 24.x.
3. Run `npm install`, an optional build script, and an optional test script in `sprint2/src`.
4. Upload `sprint2/src` as the `node-app` artifact.
5. Download the artifact in a separate deployment job.
6. Authenticate to Azure using repository secrets and OIDC.
7. Deploy the artifact to the `Team1-MessengerApp` production slot.

The staging workflow is `.github/workflows/stagingenv_team1-messengerapp-staging.yml` and follows the same process for pushes to `StagingEnv`, deploying to `Team1-MessengerApp-Staging`. The workflow does not currently trigger on pull requests. The package has no build or automated-test script, so the `--if-present` commands currently skip those phases.

Deployment

Production: Messenger v0.2 on Azure App Service.

Deployment configuration must provide `MONGO_URI` in **Azure App Service → Configuration → Application settings**. Database credentials must never be placed in `server.js`, `messengerdb.js`, a workflow file, or any committed `.env` file. Pushing an approved change to `main` triggers production deployment; `workflow_dispatch` supports a manual redeploy from GitHub Actions.

Testing and Quality Assurance

Test Strategy

The current repository relies on source review and manual functional testing across two or more browser sessions. Unit tests should isolate validation/database functions, integration tests should exercise Socket.IO events with a test database, and end-to-end tests should cover registration through logout on staging before production deployment.

On August 4, 2026, `node --check` completed successfully for `server.js`, `ui/client.js`, and `Database Files/messengerdb.js`. `npm run test --if-present` also exited successfully, but it ran no tests because `package.json` has no `test` script. Consequently, numeric code coverage is not available and the CI test phase should not be interpreted as automated verification.

Manual QA Matrix

Test	Expected result	Current evidence/result
Open production URL	Login UI loads over HTTPS; chat is hidden.	HTTP 200 verified; UI separation confirmed by source audit.
Submit invalid credentials	Login remains visible and shows a generic error.	Implemented in `join-chat`; live account test remains part of the final demo check.
Register a valid unique account	Password is hashed, account is stored, and new credentials can log in.	Client/server/data flow and bcrypt call confirmed by source audit.
Submit malformed registration or object payloads	Request is rejected without a database operator reaching a query.	Repeated format/type guards confirmed by source audit.
Send a public message between two logged-in browsers	Both receive it immediately; it is stored and returns after relogin.	Socket broadcast and `MessageHistory` insert/load path confirmed by source audit.
Send a private message	Only sender and selected online recipient receive it.	Target-socket routing confirmed by source audit.

Test	Expected result	Current evidence/result
Join the same group from two browsers	Both receive room messages; nonmembers do not.	Socket.IO room membership and room-only emit confirmed by source audit.
Edit profile with the correct current password	Selected fields update; a new password is re-hashed.	Current-password comparison and allowlisted update confirmed by source audit.
Delete an owned public message	Content becomes “This message was deleted”; another username cannot delete or spoof its deletion.	Database ownership comparison exists, but the server currently broadcasts a delete event even when no owner update occurred; hardening and a two-user live test are required.
Log out	User leaves group/list, protected UI is cleared, and login returns.	Logout cleanup confirmed by source audit.
Inspect response headers	CSP header is present.	Verified on live Azure response, August 4, 2026.
Send hostile HTML/XSS payload	Payload should display safely without executing or modifying the DOM.	**Not fully satisfied:** CSP is present, but message HTML is not sanitized before `innerHTML`.

Coverage and Next Test Priorities

Current automated coverage: **not measured**. Highest-priority additions are registration/login unit tests, bcrypt hash assertions, object/array NoSQL-injection cases, authorization tests for every socket event, message ownership tests, DOMPurify/XSS regression cases, rate-limit tests, and an end-to-end staging smoke test integrated into the workflow.

GenAI Usage and Reflection

The team used OpenAI Codex on August 4, 2026 for a documentation-focused audit. The substantive prompt asked Codex to update the Sprint 2 README using the final submission requirements, address Sprint 1 feedback, and inspect other repository files for implementation evidence.

Item	Reflection
Output accepted	Organization of the final report, requirement-to-code traceability, architecture/data descriptions, security review, implementation/CI documentation, QA matrix, and first-time-user instructions.
Output modified or reviewed	Team-specific dates, roles, contribution hours, issue status, deployment details, screenshots, demo links, and any statement that depends on private GitHub/Azure/MongoDB state require human verification.
Output rejected	Unsupported claims that DOMPurify, rate limiting, automated coverage, private/group history persistence, or a code-created unique username index already exist. No generated application source code was accepted in this documentation task.
Quality impact	The audit made the report consistent with v0.2 source and replaced template placeholders with traceable evidence.
Security impact	The audit identified the difference between CSP defense in depth and actual message sanitization, recorded the historical credential exposure, and exposed missing rate limits/tests rather than masking them.
Productivity impact	Repository-wide source, workflow, asset, and commit review was condensed into a structured final report; the team remains responsible for validating every claim and live flow before submission.

Exact prompt used: `"I need help updating my readme README.md

Here Is the entire sprint 2 submission requirments: (sprint 2 submission requirements were pasted)

Here are the comments left on the sprint 1 submission: Comments on that:

No revision history (should have at least a little bit here)

Live Prototype link is still TODO

Only 2 FR listed under user requirements (though you implemented all 9)

Implementation section not updated

Mason Hall , Jul 13 at 2:35pm I'm also not seeing any code related to CSP headers, XSS attack mitigation.

Mason Hall , Jul 13 at 2:39pm

(-2) Missing roadmap screenshot

(-1) TODO live app link, revision history mostly blank

(-1.5) Only 2 FR listed under user requirements

(-3) Implementation section not updated

(-1) Security implementation incomplete

(-3) No user guide

Some of these may have been updated but please further refine them to meet out template and requirments. You may read other files in this project to help you fill out the readme"`

Commit Link

This GenAI assistance did not replace team authorship, testing, or code review. Team members must review the report and retain responsibility for the submitted work.

Software Process Management

Scrum Process

The team used GitHub Issues as PBIs and GitHub Projects as the shared backlog. Sprint planning selected issues and assigned owners; members moved work from Todo to In Progress to Done and documented progress in issue comments. Development occurred on feature or staging branches, then moved through pull requests and merges. Each sprint concluded with a review/demo, contribution summary, and retrospective. The Scrum Master coordinated ceremonies and board hygiene, while the Product Owner prioritized requirements and acceptance criteria.

Sprint 0

Duration: May 27–June 16, 2026

Sprint goal: Define the Messenger problem, users, requirements, use cases, security concerns, architecture, and team delivery process.

Completed PBIs and Tasks

1. Established the organization, repositories, team homepage, roles, and GitHub Project.
2. Created personas, scenarios, user stories, use cases, and acceptance criteria.
3. Created sequence diagrams and UI mockups.
4. Documented architecture, security requirements, trust boundaries, and STRIDE threats.
5. Added the initial Messenger HTML and completed the sprint review/retrospective.

Contributions

Member	Hours	Contribution summary and commit evidence
Corey Brunner	7	Added UC-01/UC-02, acceptance criteria, diagrams, UI designs, basic UI, and board coordination. Representative commit `8a27eed`
Pawan Poudyel	6	Added the external-links use case, NFR, threat model content, diagram references, and user-story subissues. Representative commit `06b133b`
Zackary Bailey	6	Added the status use case, PBIs, acceptance criteria, sequence diagram, and retrospective. Representative commit `e4aa797`
Abdulah Dia	6	Added the architecture diagram, SSDLC text, user story, and sequence-diagram work. Representative commit `917352f`
Garrett Becker	6	Added the group-message use case, acceptance criteria, security requirements, and board updates. Representative commit `8ee09f6`

Sprint Retrospective

Went well	Could improve	Sprint 1 action
Team coordination and use-case definition	Project-board organization	Communicate issue ownership and required fields before work begins.
Acceptance criteria and diagrams	Duplicate/overlapping PBIs	Review new PBIs against existing items before adding them.

Sprint 1

Duration: June 17–July 9, 2026

Sprint goal: Deliver and deploy Messenger v0.1 with separated login/chat views, live public/private messaging, presence, typing indicators, and external links.

Completed PBIs and Tasks

1. Change the composer to a contenteditable element for links (#38)
2. Automatically convert pasted/typed URLs (#30)
3. Add a link-insertion button (#31)
4. Visually separate public and private chat, F1.7 (#43)
5. Add public/private typing status, F1.8 (#44)
6. Added username entry, authenticated-view separation, real-time public/private messaging, presence, auto-scroll, and Azure deployment.

Contributions

Member	Hours	Contribution summary and commit evidence
Corey Brunner	6	Created and debugged Azure App Service deployment and implemented the separated login/chat flow and username display. Representative commit `ebdc895`
Pawan Poudyel	6	Implemented external-link user stories and updated overview, system analysis, backlog, and use-case realization. Representative commit `b78e843`
Zackary Bailey	6	Implemented/upgraded the online-user display, CSS, PBIs, and retrospective documentation. Representative commit `75db382`
Abdulah Dia	6	Worked on visual separation and typing-indicator requirements and related PBIs. Representative commit `11157be`
Garrett Becker	6	Implemented public auto-scroll and Socket.IO direct messaging; updated system design and PBIs. Representative commit `1e968ec`

Sprint Retrospective

Went well	Could improve	Sprint 2 action
Distribution of implementation work, code quality, and documentation	Scheduling and inconsistent communication	Schedule recurring meetings at the start of the sprint.

Went well	Could improve	Sprint 2 action
Working public/private prototype and deployment	Late alignment of member availability	Use consistent team messages and raise blockers earlier.

Sprint 2

Duration: July 15–July 30, 2026

Sprint goal: Deliver Messenger v0.2 with database-backed accounts and history, secure credential handling, profile/logout flows, a refined UI, group communication, and production/staging deployment.

Completed PBIs and Tasks

1. Connected the application to MongoDB Atlas and configured `MONGO_URI` locally and in Azure.
2. Implemented registration, bcrypt password authentication, and logged-in-only operations.
3. Added profile editing, logout, online-user profile viewing, and public chat history.
4. Added owner-only soft deletion of public messages.
5. Added name-based group chat and repaired private messaging/typing behavior.
6. Updated the Bootstrap login/registration interface and public/private/group panels.
7. Added CSP response headers, defense-in-depth validation, staging deployment, diagrams, and report updates.

Contributions

Member	Hours	Contribution summary and commit evidence
Corey Brunner	6	Set up MongoDB/Azure staging work, account-management SPA behavior, Bootstrap authentication UI, validation improvements, PBIs, and CI/CD changes. Representative commit `befef78`
Pawan Poudyel	6	Implemented database login/registration and history, online-profile viewing, private-link fixes, diagrams/PBIs, and README updates. Representative commit `123c81f`
Zackary Bailey	6	Implemented owner-facing public-message deletion, group-chat UI/CSS, user-list fixes, PBIs, and retrospective. Representative commit `4f1826b`

Member	Hours	Contribution summary and commit evidence
Abdulah Dia	6	Implemented group tracking and messaging, UI improvements, and fixes for private messaging and typing indicators. Representative commit `10fd75d`
Garrett Becker	6	Moved MongoDB credentials to environment configuration and implemented profile/password editing, email/full name, logout, PBIs, and security documentation. Representative commit `67c62a3`

Sprint Retrospective

Went well	Could improve	Final integration action
Feature implementation and staging-to-production CI/CD	Work ordering and distribution	Identify dependencies and owners during planning before coding begins.
MongoDB/account integration and UI refinement	Scheduling and communication	Establish recurring meetings and raise blockers in the shared channel.
Team members contributed distinct v0.2 features	Security/testing work arrived late	Add security acceptance criteria and test tasks to each PBI from the start.

User Guide and Demo

Live Application and Recordings

- Open Messenger v0.2 on Azure
- Team homepage and demo archive
- Sprint 1 demonstration recording

First-Time User Guide

1. Register an Account

1. Open the live application and select **Register Here**.

2. Enter a username containing 3–20 letters, numbers, or underscores.
3. Enter a password of at least six characters containing at least one letter and one number.
4. Enter a valid email address and full name.
5. Select **Register Account**. A duplicate or invalid entry displays an inline error; a successful registration returns to login.

2. Log In

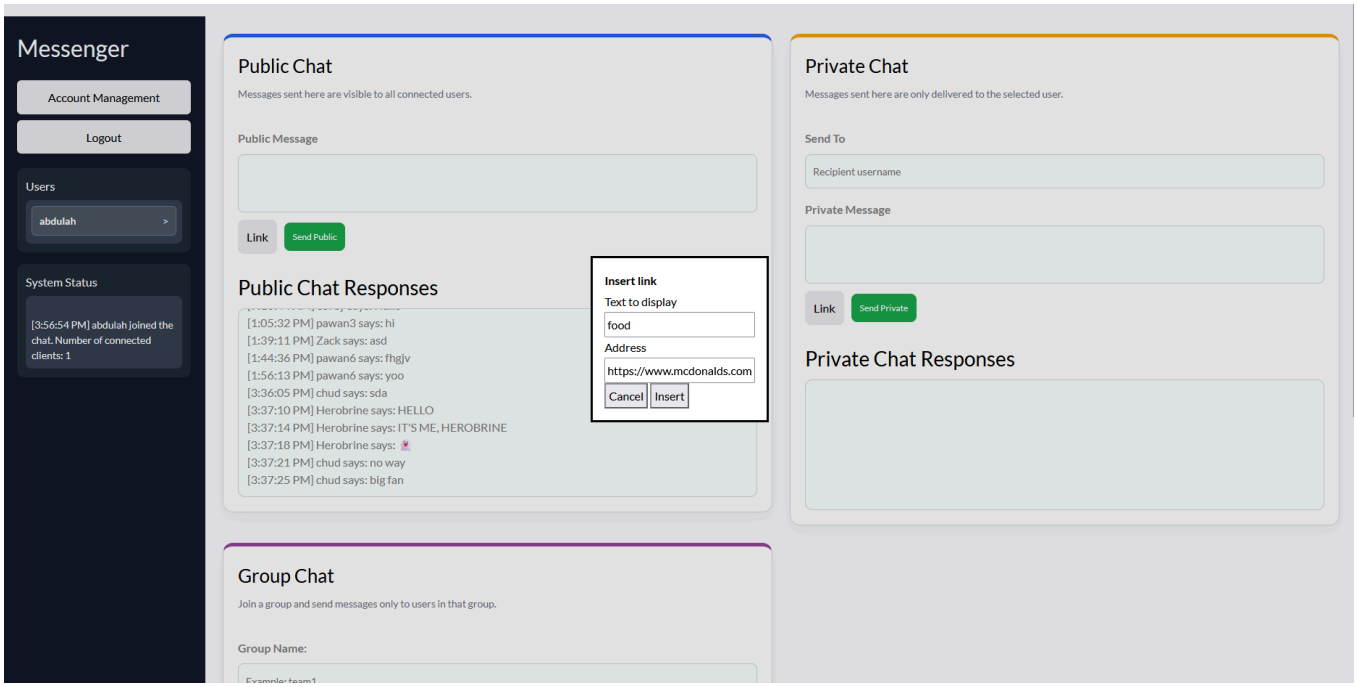
1. Enter the registered username and password on the login page.
2. Select **Login**.
3. Valid credentials open the chat UI and load recent public history. Invalid credentials leave the login page visible and show a generic error.

3. Send and Read Public Messages

1. Enter text in **Public Message**.
2. Select **Send Public** or press Enter.
3. The message appears for every authenticated connected user and is stored in public history.
4. After a later login, the latest 100 public messages load automatically.

4. Add an External Link

Paste or type a full `http://` or `https://` address; it becomes clickable automatically. For custom display text, select **Link**, enter the text and address, then select **Insert**. Links open in a new browser tab.

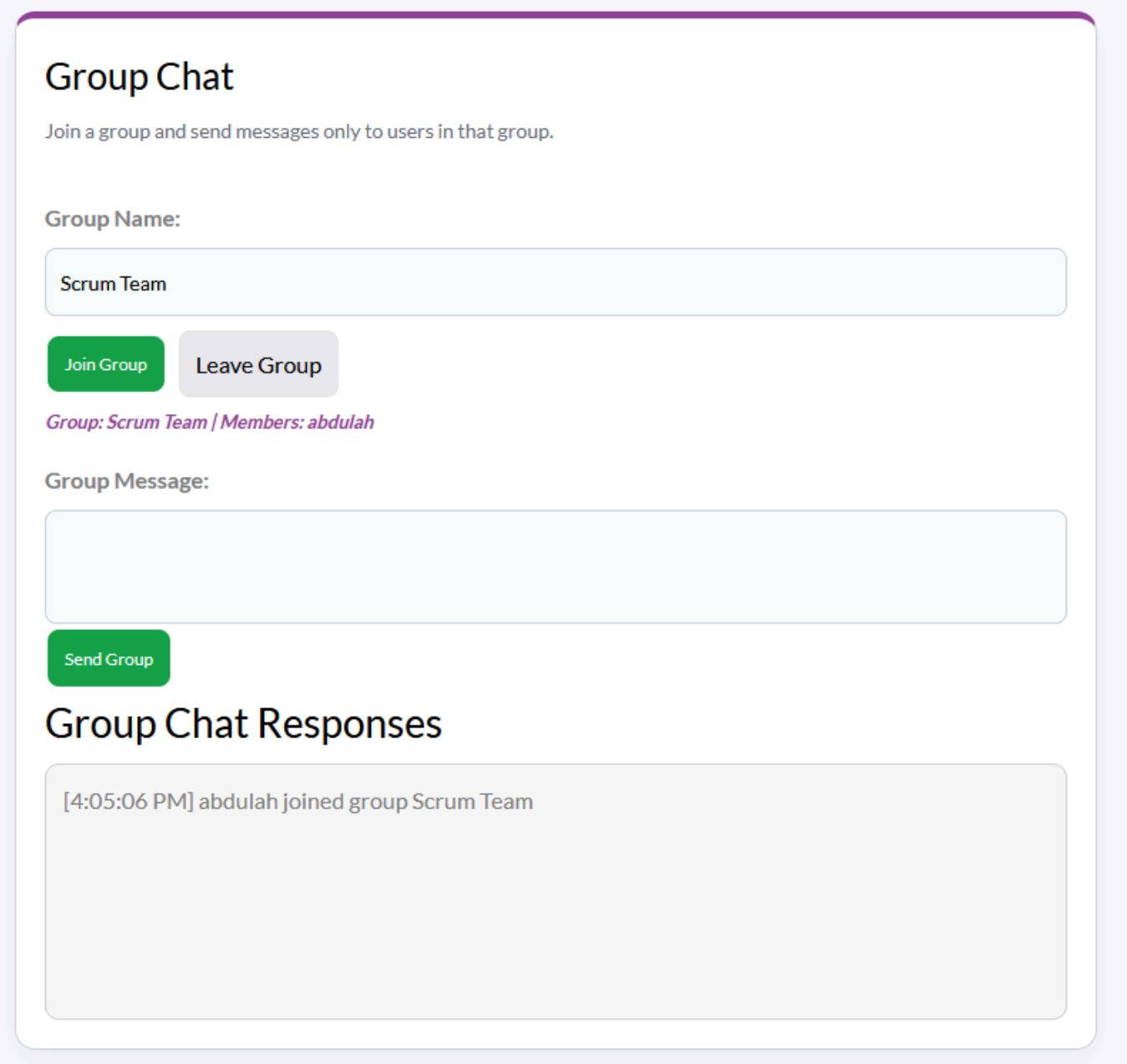


5. Send a Private Message

1. Find the intended recipient in the **Users** sidebar and enter that exact username in **Send To**.
2. Enter the message in **Private Message**.
3. Select **Send Private** or press Enter.
4. Only the sender and online recipient receive the message. Private history is not retained after the connection ends.

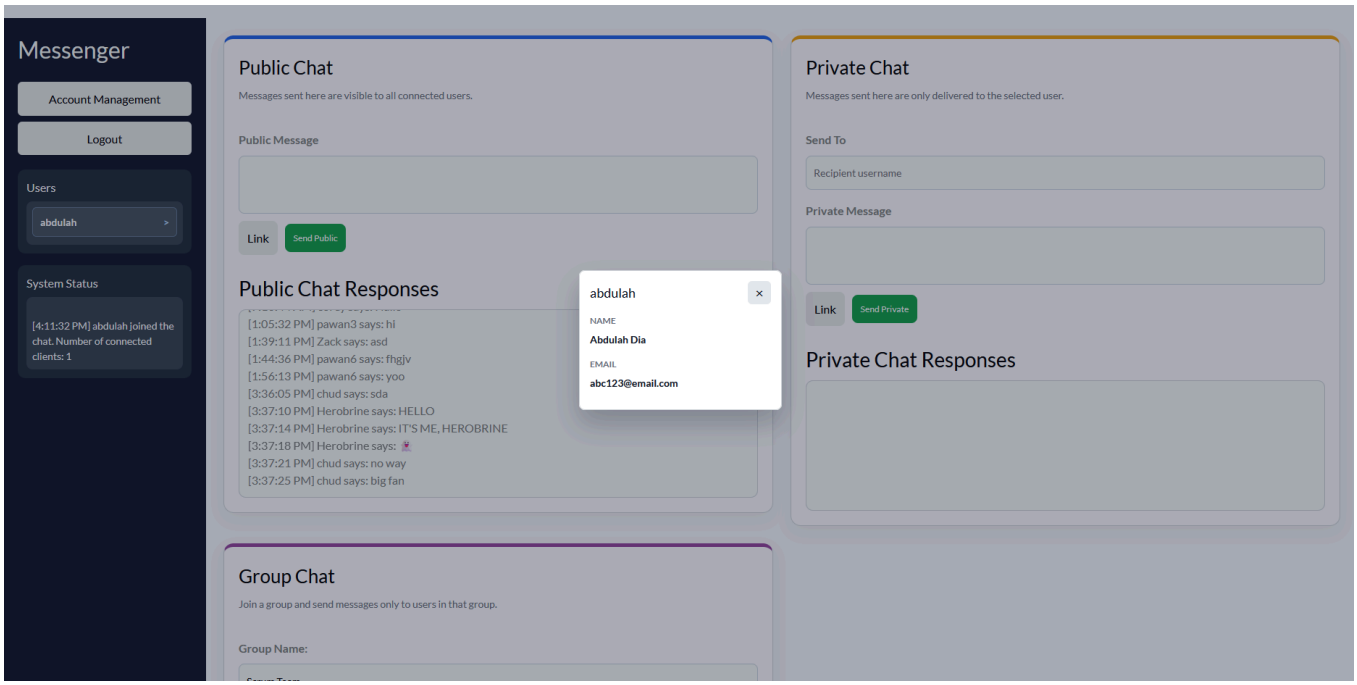
6. Join and Use a Group

1. Agree on a group name with the other participants.
2. Enter the same value in **Group Name** and select **Join Group**.
3. Enter text and select **Send Group**. Only current members of that Socket.IO room receive it.
4. Select **Leave Group** when finished. A user can be in one active group at a time, and group history is not persisted.



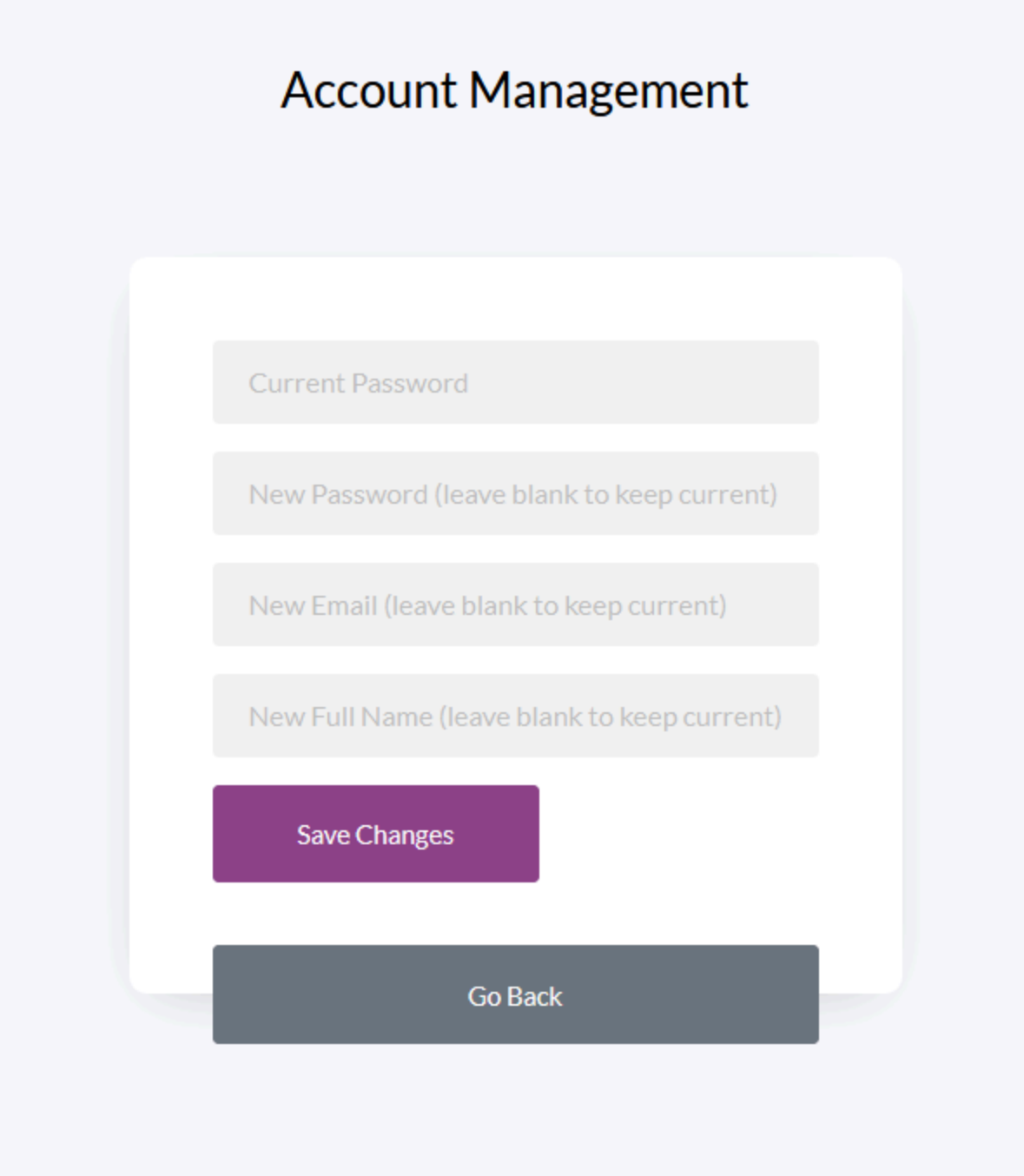
7. View an Online User's Profile

Select a username in the **Users** list. The profile dialog shows the user's username, full name, and email. Close the dialog with the `×` button. Only currently online users can be queried.



8. Edit Your Profile or Password

1. Select **Account Management**.
2. Enter the current password.
3. Enter one or more new values: password, email, or full name.
4. Select **Save Changes**. A password change must still contain letters and numbers and is stored as a new bcrypt hash.
5. Select **Go Back** to return to chat.



9. Delete Your Own Public Message

Select the `x` next to one of your public messages. The database retains the record for conversation continuity but replaces its content with **This message was deleted**. Other users cannot delete that message.

10. Log Out

Select **Logout**. Messenger removes the socket from the online list and active group, clears the visible chat state, and returns to the login screen. Close the browser as an additional precaution on a shared computer.

License and Code of Conduct

The repository includes the Apache License 2.0. This project was developed for academic purposes in EECE/CS-3093C at the University of Cincinnati. Team members follow the ACM Code of Ethics and Professional Conduct and the course academic-integrity policy.

Last template revision: 2026-08-04